

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ МОНИТОРИНГА ТАРИФОВ

2.1 Общая архитектура системы и контекстная модель

Проектирование архитектуры разрабатываемой системы выполнено на основе требований, сформулированных в разделах 1.3 и 1.5 первой главы пояснительной записки. К числу ключевых требований относятся: обновление тарифных данных не реже одного раза в пять минут, время ответа REST API не более 200 мс, одновременная работа не менее 50 пользователей, доступность 99,9 % в рабочее время, ведение истории тарифов глубиной не менее 12 месяцев, аутентификация на основе JWT с ролевой моделью, экспорт данных в форматах CSV и Microsoft Excel, поддержка webhook-уведомлений и наличие документированного REST API.

Архитектура системы спроектирована в соответствии с моделью C4 [12], предложенной С. Брауном для структурного описания программных систем. Согласно данной модели, описание начинается с верхнего уровня — контекстной схемы, отражающей взаимодействие системы с пользователями и внешними системами. Контекстная схема разрабатываемой системы представлена на рисунке 2.1.

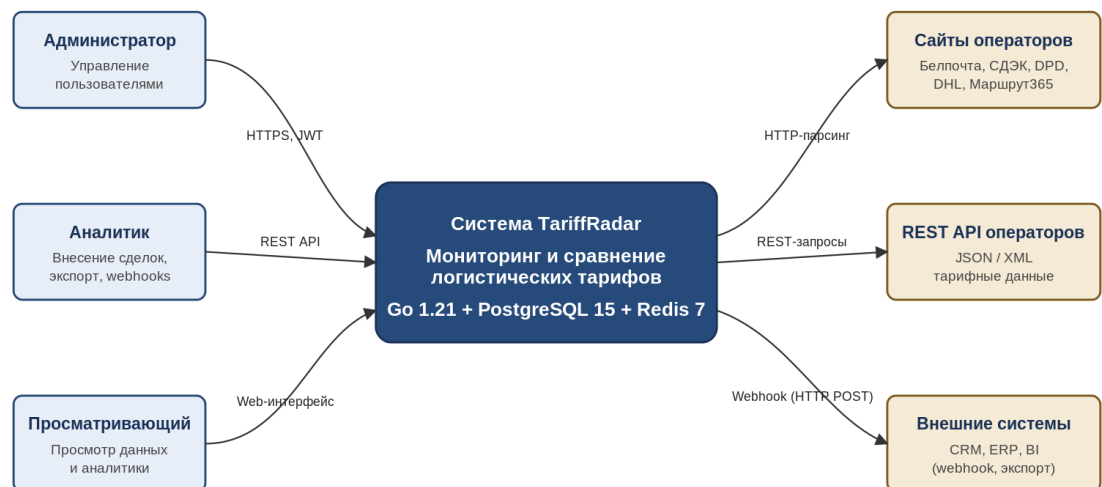


Рисунок 2.1 – Контекстная схема системы TariffRadar

Из рисунка 2.1 следует, что система TariffRadar взаимодействует с тремя категориями пользователей и с тремя группами внешних систем. Пользователи системы разделены по ролям в соответствии с моделью управления доступом,

описанной в подразделе 2.7: администратор обеспечивает управление учётными записями, аналитик имеет полномочия на внесение сделок, экспорт данных и регистрацию webhook-уведомлений, просматривающий обладает правом доступа только к аналитическим экранам. Внешние системы подразделяются на источники данных (официальные сайты и REST API логистических операторов) и системы-получатели уведомлений (внешние CRM-, ERP- и BI-системы предприятия-заказчика).

Контекстная модель определяет границы разрабатываемой системы и её внешние интерфейсы. На втором уровне модели C4 проводится детализация внутренней структуры системы в виде схемы контейнеров (автономно развёртываемых компонентов). Архитектура контейнеров разрабатываемой системы представлена на рисунке 2.2.

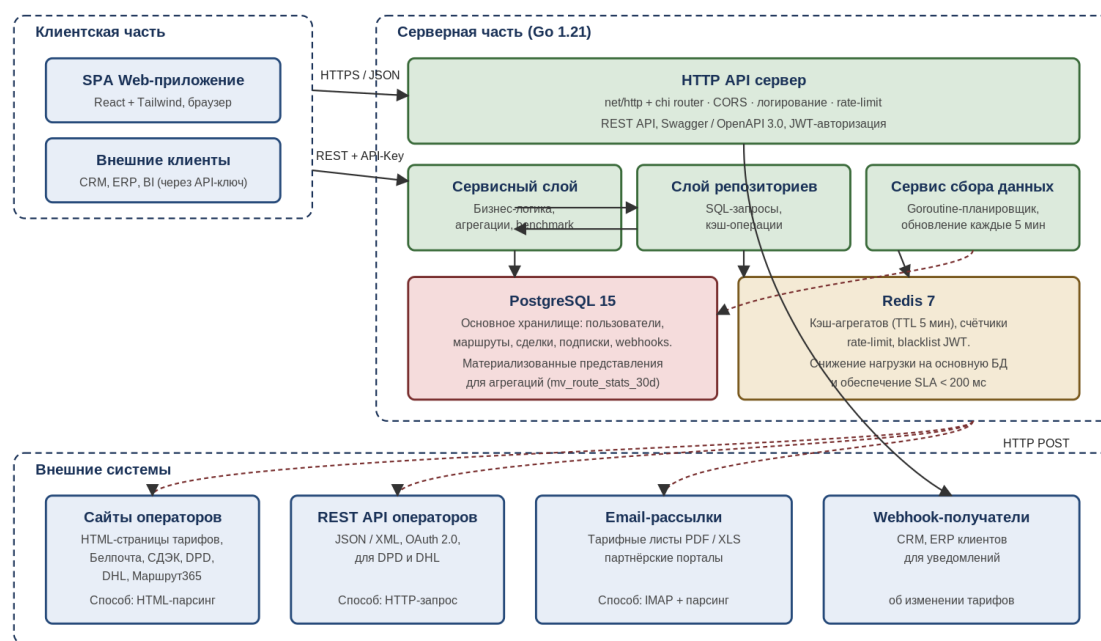


Рисунок 2.2 – Архитектура контейнеров системы

В соответствии с рисунком 2.2 разрабатываемая система состоит из четырёх логически обособленных контейнеров, выполняющих самостоятельную функциональную роль. Контейнер «HTTP API сервер» реализован на языке Go и обеспечивает обработку всех запросов клиентских приложений, валидацию входных данных, формирование ответов в формате JSON и применение middleware-цепочек (аутентификация, ограничение скорости, логирование). Контейнер «Сервис сбора данных» функционирует независимо от HTTP-сервера и

в фоновом режиме осуществляет периодический опрос источников тарифных данных, парсинг полученной информации и её сохранение в основную базу данных. Контейнер PostgreSQL 15 представляет собой основное хранилище данных, в котором размещаются как нормализованные таблицы, так и материализованные представления, используемые для предрасчётной агрегации. Контейнер Redis 7 выполняет роль кэша агрегатов и сервисных счётчиков (rate-limiting, blacklist JWT-токенов).

Указанное разделение продиктовано необходимостью изоляции долговременных операций сбора данных от обработки пользовательских запросов с жёстким SLA по времени отклика. Использование общей кодовой базы для HTTP-сервера и сервиса сбора данных при их отдельном развёртывании в виде обособленных процессов обеспечивает повторное использование репозиторного слоя и доменных моделей.

2.2 Слоистая архитектура серверной части

Внутренняя структура серверной части построена в соответствии с принципами Clean Architecture, изложенными Р. Мартином в работе [5]. В основу подхода положено правило направления зависимостей: код, реализующий бизнес-логику, не должен зависеть от внешних деталей реализации (фреймворков, СУБД, протоколов передачи данных). Связь между слоями осуществляется посредством абстракций — интерфейсов языка Go, что обеспечивает независимое тестирование и возможность замены реализации каждого из слоёв без модификации остальных.

Серверная часть разработана в виде четырёхуровневой архитектуры. Структурная схема слоёв и их взаимодействия представлена на рисунке 2.3.

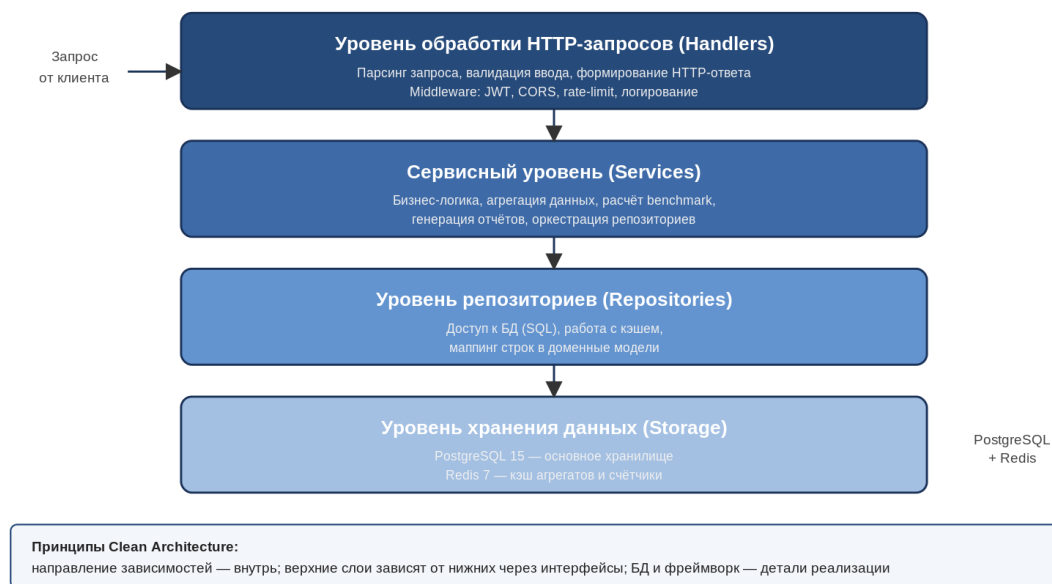


Рисунок 2.3 – Слоистая архитектура серверной части

Согласно рисунку 2.3, серверная часть включает четыре уровня: уровень обработки HTTP-запросов (handlers), сервисный уровень (services), уровень репозитория (repositories) и уровень хранения данных (storage). Распределение функциональной ответственности между уровнями приведено в таблице 2.1.

Таблица 2.1 – Распределение ответственности между уровнями серверной части

Уровень	Назначение	Ключевые компоненты
Handlers	Приём HTTP-запросов, парсинг параметров, формирование ответов, применение цепочки middleware	auth, market, deals, benchmark, insights, my_routes, webhooks, export
Services	Реализация бизнес-логики, оркестрация репозитория, агрегация данных, расчёт benchmark-показателей	AuthService, MarketService, BenchmarkService, InsightsService, ExportService, CollectorService
Repositories	Доступ к базе данных и кэшу, маппинг строк в доменные модели, инкапсуляция SQL-запросов	UserRepo, DealRepo, MarketRepo, BenchmarkRepo, InsightsRepo, UserRouteRepo, WebhookRepo

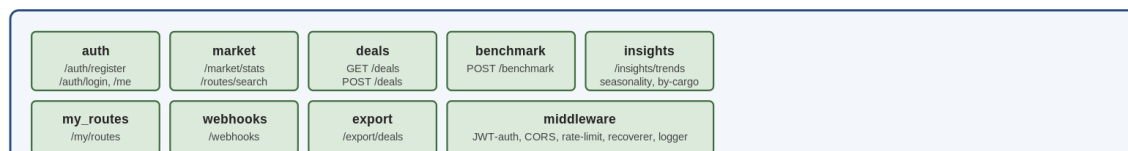
Storage	Физическое хранение данных и кэшированных значений	PostgreSQL 15, Redis 7
---------	---	------------------------

Принципиальной особенностью спроектированной архитектуры является односторонняя зависимость уровней. Так, уровень handlers зависит от сервисного уровня, но не имеет прямого доступа к репозиториям или СУБД. Сервисный уровень оперирует абстрактными интерфейсами, реализации которых предоставляются репозиторным уровнем. Уровень репозитория инкапсулирует все детали работы с PostgreSQL и Redis, что позволяет, например, заменить СУБД без изменения бизнес-логики. Подобная организация облегчает написание модульных тестов: каждый уровень может быть протестирован независимо, с применением подменных (mock) реализаций соседних уровней.

2.3 Модульная декомпозиция серверной части

Внутри каждого из четырёх уровней, описанных в подразделе 2.2, выделены функциональные модули, соответствующие логически обособленным частям предметной области. Принципом декомпозиции послужило соответствие модулей экранам пользовательского интерфейса и сценариям использования системы. Модульная структура серверной части представлена на рисунке 2.4.

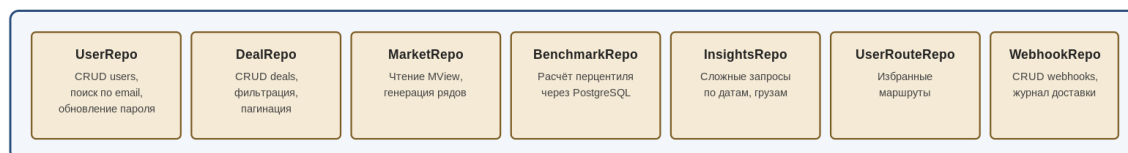
Слой Handlers — обработчики REST API (по экранам клиента)



Сервисный слой — бизнес-логика



Слой репозитория — доступ к данным



Слой хранения данных



Рисунок 2.4 – Модульная декомпозиция серверной части

Согласно рисунку 2.4, выделено восемь функциональных модулей: auth, market, deals, benchmark, insights, my_routes, webhooks, export. Каждый модуль присутствует одновременно на трёх верхних уровнях (handlers, services, repositories). Назначение модулей описано в таблице 2.2.

Таблица 2.2 – Функциональные модули серверной части

Модуль	Назначение	Ключевые операции
auth	Управление учётными записями и аутентификацией	Регистрация, вход, выход, выдача и валидация JWT, blacklist токенов
market	Получение рыночных показателей по маршруту	Расчёт KPI (средняя/мин/макс ставка), формирование временных рядов
deals	Работа со сделками	Просмотр анонимизированной истории, добавление новых сделок аналитиком
benchmark	Сравнение пользовательской ставки с рынком	Расчёт перцентиля, отклонения, формирование рекомендации
insights	Аналитические отчёты	Топ растущих/падающих маршрутов, сезонность, сравнение по типам грузов
my_routes	Управление избранными маршрутами	Добавление/удаление маршрутов из списка избранного
webhooks	Уведомления при изменении тарифов	Регистрация webhook'ов, фильтрация событий, доставка с повторными попытками
export	Выгрузка данных во внешние форматы	Генерация CSV и XLSX, потоковая запись больших выборок

Модульная декомпозиция обеспечивает изолированность сценариев использования: внесение новой функциональности в один из модулей не требует изменений в остальных. Кроме того, такая структура упрощает распределение задач между разработчиками и формирование плана поэтапной разработки, описанного в третьей главе пояснительной записки.

2.4 Проектирование модели данных

Проектирование структуры базы данных выполнено в три этапа: построение концептуальной модели в нотации «сущность—связь», переход к логической модели для реляционной СУБД и определение физических аспектов реализации (типов данных, индексов, ограничений целостности). При выборе структуры таблиц учитывались требования к скорости агрегационных запросов, объёму хранимых данных и возможности горизонтального масштабирования.

Логическая ER-диаграмма основных сущностей разрабатываемой системы приведена на рисунке 2.5.

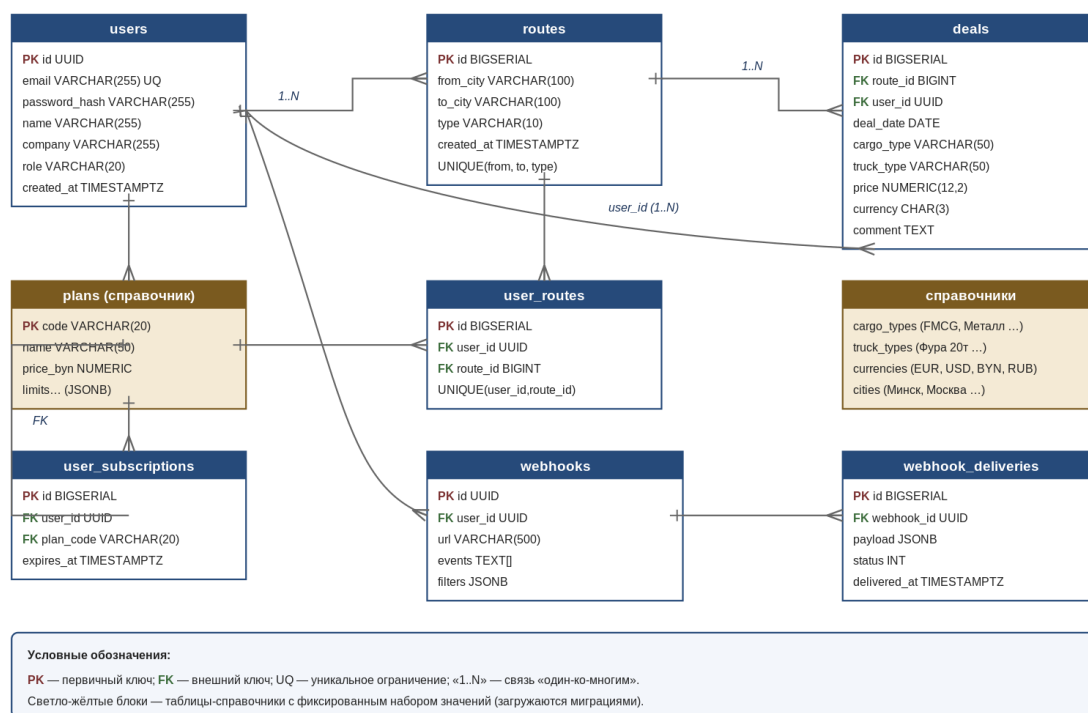


Рисунок 2.5 – ER-диаграмма базы данных

Из рисунка 2.5 следует, что центральной сущностью модели является таблица deals («сделки»), хранящая факты выполненных перевозок. Каждая сделка связана со справочной сущностью routes («маршруты») по внешнему ключу route_id и с пользователем-инициатором по внешнему ключу user_id. Таблица routes выделена в отдельную сущность с целью устранения избыточности при хранении повторяющихся комбинаций «город отправления — город назначения — тип перевозки» и для возможности построения индексов исключительно по идентификатору маршрута.

Тип данных для денежных значений (поле price таблицы deals) выбран как NUMERIC(12,2), что обеспечивает точное представление сумм без накопления

ошибок округления, характерных для типов с плавающей точкой. Идентификаторы пользователей реализованы в виде UUID, что упрощает распределённую генерацию идентификаторов и не раскрывает внешним сторонам количество зарегистрированных учётных записей. Для сделок и маршрутов применён тип BIGSERIAL, позволяющий сохранять монотонно возрастающую нумерацию и эффективно использовать кластеризованные индексы.

Сущности plans и user_subscriptions реализуют тарификацию доступа к системе. Справочник plans содержит описание тарифных планов (Free, Pro, Enterprise) и их количественных ограничений (максимальное число избранных маршрутов, разрешённое число webhook'ов, лимит запросов API в час). Сущности webhooks и webhook_deliveries предназначены для регистрации внешних адресов уведомлений и журналирования попыток доставки соответственно.

Перечень основных таблиц базы данных и их назначения приведён в таблице 2.3.

Таблица 2.3 – Перечень основных таблиц и материализованных представлений

Таблица	Назначение	Размер записи (оценочно)
users	Учётные записи пользователей системы	~ 350 байт
routes	Справочник уникальных маршрутов	~ 230 байт
deals	Сделки (факты перевозок)	~ 180 байт
user_routes	Избранные маршруты пользователей	~ 60 байт
plans	Справочник тарифных планов	~ 200 байт
user_subscriptions	Активные подписки пользователей	~ 100 байт
user_notifications	Настройки уведомлений (1:1 с users)	~ 80 байт
webhooks	Зарегистрированные webhook-адреса	~ 600 байт
webhook_deliveries	Журнал доставки webhook-уведомлений	~ 800 байт
mv_route_stats_30d	Материализованное представление: агрегаты за 30 дней	кэш
mv_route_series_3	Материализованное	кэш

0d	представление: ряды за 30 дней	
----	--------------------------------	--

Особое внимание при проектировании уделено индексам таблицы `deals`, поскольку именно эта таблица является основным источником данных для аналитических запросов и обладает наибольшим темпом роста. Согласно оценке нагрузки, при средней интенсивности 200–400 новых сделок в сутки за двенадцать месяцев в таблице будет накоплено от 70 до 150 тыс. записей, а при дальнейшем росте числа клиентов системы — до нескольких миллионов. Для обеспечения времени аналитических запросов не более 200 мс предусмотрены следующие индексы:

- `idx_deals_route_date` — составной индекс по полям (`route_id`, `deal_date` DESC), обслуживающий типовой запрос «свежие сделки по конкретному маршруту»;
- `idx_deals_date` — индекс по полю `deal_date` DESC для общих выборок и формирования сезонных рядов;
- `idx_deals_user` — индекс по полю `user_id` для отображения сделок текущего пользователя.

Дополнительно для обеспечения масштабируемости предусмотрена возможность партиционирования таблицы `deals` по полю `deal_date` (диапазонное по месяцам). На этапе MVP партиционирование не реализуется, однако структура таблицы спроектирована таким образом, чтобы переход к партиционным таблицам не требовал изменений в репозиторном слое.

Для соблюдения SLA по времени отклика API применены материализованные представления `mv_route_stats_30d` и `mv_route_series_30d`, содержащие агрегированные показатели по маршрутам. Обновление представлений выполняется командой `REFRESH MATERIALIZED VIEW CONCURRENTLY` каждые пять минут, что соответствует требованию ТЗ к актуальности данных. Применение `CONCURRENTLY` обеспечивает обновление без блокировки конкурентных запросов на чтение.

2.5 Проектирование REST API

Программный интерфейс системы спроектирован в соответствии с архитектурным стилем REST (Representational State Transfer), описанным Р. Филдингом и подробно изложенным в работе [10]. Соблюдение принципов REST обеспечивает единообразие взаимодействия между клиентом и сервером, независимость от состояния (stateless) и кэшируемость ответов.

При проектировании API соблюдены следующие принципы: ресурсо-ориентированная адресация (каждая URL-сущность соответствует одному типу ресурсов), использование стандартных HTTP-методов для операций (GET для чтения, POST для создания, PATCH для частичного обновления, DELETE для удаления), применение унифицированного формата представления данных JSON и единого формата ошибок, явное указание версии API в составе адресной строки (/api/v1).

Все ресурсы API сгруппированы в семейства, соответствующие функциональным модулям, описанным в подразделе 2.3. Сводный перечень основных групп эндпоинтов представлен в таблице 2.4.

Таблица 2.4 – Сводный перечень эндпоинтов REST API

Группа	Метод	Путь	Назначение
Auth	POST	/auth/register	Регистрация нового пользователя
Auth	POST	/auth/login	Аутентификация и выдача JWT
Auth	GET	/me	Профиль текущего пользователя
Market	GET	/market/stats	KPI и временные ряды по маршруту
Market	GET	/routes/search	Поиск маршрута
Market	GET	/routes/suggestions	Автодополнение городов
Deals	GET	/deals	Анонимизированная история сделок
Deals	POST	/deals	Внесение новой сделки (analyst, admin)
Benchmark	POST	/benchmark	Сравнение ставки пользователя с рынком
Insights	GET	/insights/trends	Топ растущих и падающих маршрутов
Insights	GET	/insights/seasonality	Сезонность ставки по месяцам
Insights	GET	/insights/by-cargo	Сравнение по типам

hts			грузов
My Routes	GET / POST / DELETE	/my/routes	Управление избранными маршрутами
Web hooks	GET / POST / DELETE	/webhooks	Регистрация webhook- адресов
Exp ort	GET	/export/deals	Экспорт истории сделок в CSV / XLSX
Setti ngs	PATC H	/me, /me/password, /me/notifications	Настройки профиля и уведомлений
Serv ice	GET	/health	Проверка живости сервиса

Документация API оформлена в формате OpenAPI 3.0 и публикуется через интерфейс Swagger UI по адресу /docs. Использование стандарта OpenAPI обеспечивает совместимость с инструментами автогенерации клиентских библиотек и упрощает интеграцию с системами заказчика.

Для обеспечения единообразия ошибок все некорректные ответы сервера возвращаются в формате JSON-объекта со структурой {"error": {"code": "<код>", "message": "<сообщение>", "field": "<поле>"}}, где поле field присутствует только для ошибок валидации входных данных. Применение единого формата позволяет клиентскому приложению обрабатывать ошибки централизованно.

2.6 Стратегия кэширования и оптимизация производительности

Соблюдение требования к времени ответа API не более 200 мс при ожидаемой нагрузке достигается применением многоуровневой стратегии кэширования. Структура уровней кэша и их характеристики представлены на рисунке 2.6.

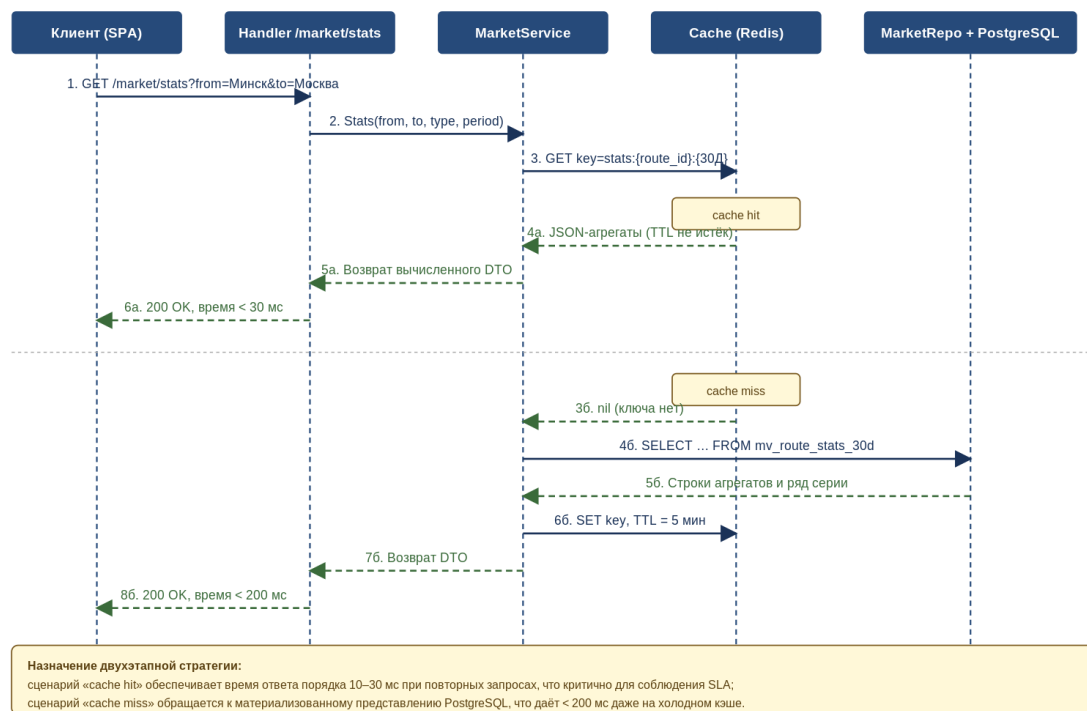


Рисунок 2.6 – Sequence-диаграмма обработки запроса GET /market/stats

Согласно рисунку 2.6, при поступлении запроса на получение рыночных показателей по маршруту сервисный слой первоначально выполняет проверку наличия предвычисленного результата в кэше Redis. В случае попадания в кэш (cache hit) ответ формируется без обращения к базе данных, что обеспечивает время ответа порядка 10–30 мс. В случае промаха (cache miss) запрос перенаправляется в репозиторийный слой, где данные читаются из материализованного представления mv_route_stats_30d, после чего сохраняются в кэш с временем жизни (TTL) в пять минут. Двухэтапный подход гарантирует выполнение SLA даже при холодном кэше — обращение к материализованному представлению занимает значительно меньше времени, чем агрегация исходной таблицы deals.

Принципиальная схема двухуровневого кэширования системы показана на рисунке 2.7.

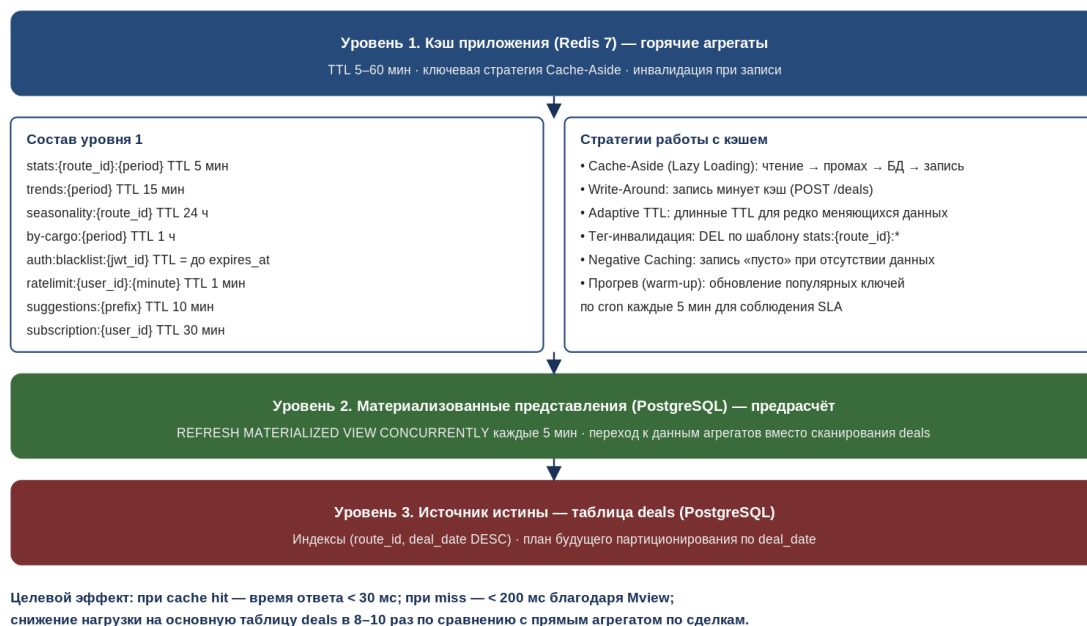


Рисунок 2.7 – Двухуровневая стратегия кэширования

В соответствии с рисунком 2.7 кэширование организовано на трёх уровнях. Первый уровень — кэш приложения в Redis, хранящий часто запрашиваемые агрегаты в виде сериализованных JSON-объектов. Второй уровень — материализованные представления PostgreSQL, обновляемые по расписанию. Третий уровень — таблица deals, являющаяся источником истины. Перечень используемых ключей кэша Redis и их характеристики приведены в таблице 2.5.

Таблица 2.5 – Стратегия кэширования: ключи Redis и их характеристики

Шаблон ключа	TTL	Назначение
stats:{route_id}:{period}	5 мин	Агрегированные показатели по маршруту за период
trends:{period}	15 мин	Топ растущих и падающих маршрутов
seasonality:{route_id}	24 ч	Сезонность ставки по месяцам
by-cargo:{period}	1 ч	Средние ставки по типам грузов
suggestions:{prefix}	10 мин	Автодополнение названий городов
subscription:{user_id}	30 мин	Текущая подписка пользователя
auth:blacklist:{jwt_id}	до exp	Чёрный список инвалидированных JWT

ratelimit: {user_id}: {minute}	1 мин	Счётчик запросов для ограничения частоты
--------------------------------	----------	--

Применяется стратегия Cache-Aside с явной инвалидацией: при добавлении новой сделки модуль deals удаляет из кэша Redis все ключи вида stats:{route_id}:* для соответствующего маршрута. Это гарантирует, что повторное обращение к /market/stats по данному маршруту приведёт к пересчёту агрегатов на актуальной выборке. Sequence-диаграмма обработки операции добавления сделки с инвалидацией кэша и инициированием webhook-уведомлений приведена на рисунке 2.8.

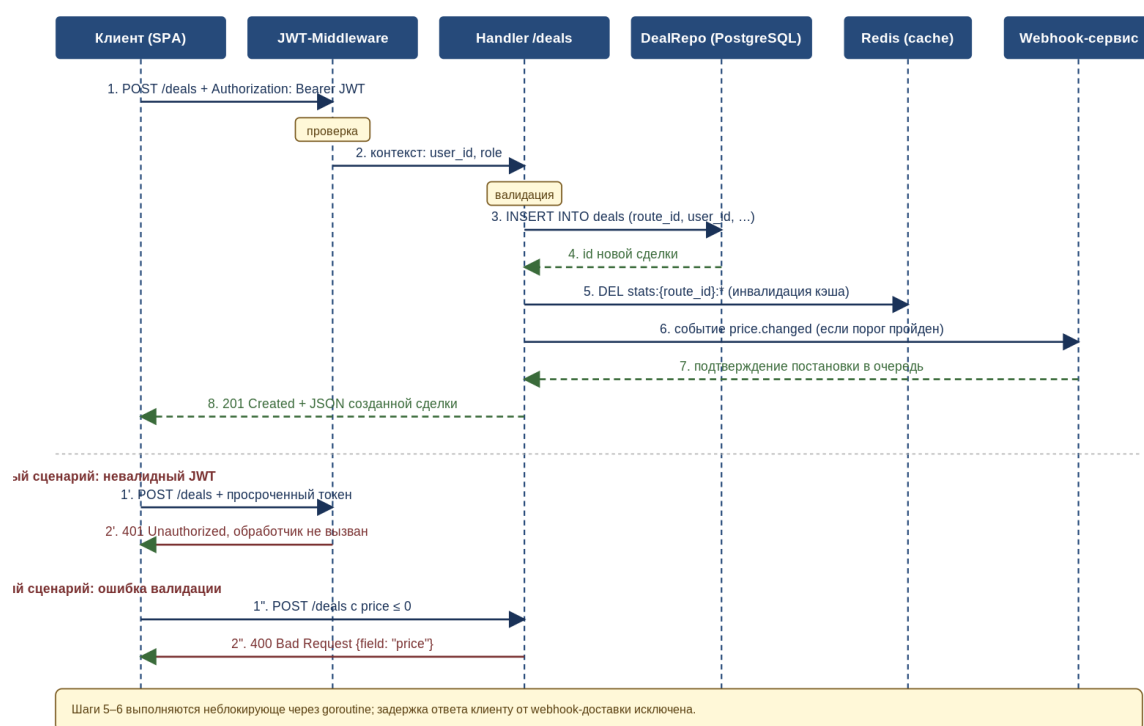


Рисунок 2.8 – Sequence-диаграмма обработки запроса POST /deals

Согласно рисунку 2.8, операция добавления сделки сопровождается выполнением трёх типов побочных действий: инвалидацией соответствующих ключей кэша, постановкой webhook-события в очередь и возвратом подтверждения клиенту. Постановка событий в очередь и доставка webhook-уведомлений выполняются асинхронно с применением goroutine-моделей языка Go, что исключает блокирование основного потока обработки запроса и обеспечивает быстрый ответ клиенту.

2.7 Безопасность системы и ролевая модель доступа

Защита данных и разграничение прав пользователей реализованы через четыре механизма: принудительное использование протокола HTTPS на уровне обратного прокси, аутентификация на основе JWT-токенов, ролевая модель доступа и ограничение частоты запросов (rate-limiting). Принципиальная схема взаимодействия указанных механизмов представлена на рисунке 2.9.

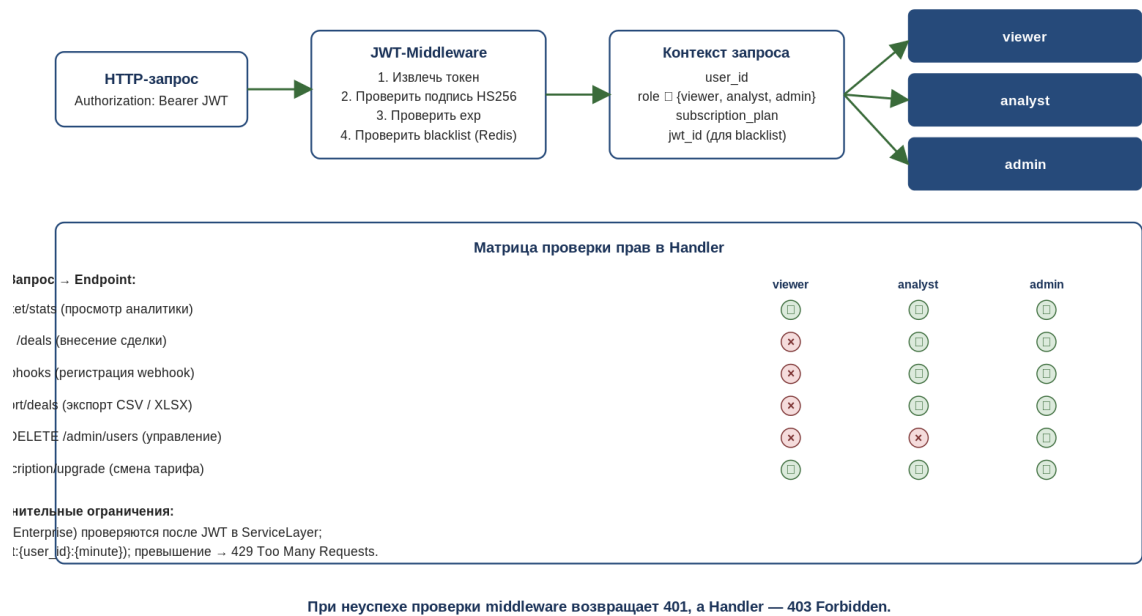


Рисунок 2.9 – Схема аутентификации и матрица доступа по ролям

В соответствии с рисунком 2.9, каждый запрос к защищённым эндпоинтам должен содержать заголовок `Authorization: Bearer <JWT>`. Проверка токена выполняется в `JWT-Middleware`, последовательно: извлекается токен из заголовка, проверяется подпись по алгоритму `HMAC-SHA256`, проверяется срок действия (`claim exp`), производится сверка с чёрным списком инвалидированных токенов в `Redis`. При успешной валидации в контекст HTTP-запроса добавляются атрибуты `user_id`, `role` и идентификатор токена; в противном случае возвращается ответ `HTTP 401 Unauthorized` без вызова обработчика.

Ролевая модель включает три предопределённые роли: `viewer` (просматривающий), `analyst` (аналитик) и `admin` (администратор). Сводная матрица прав доступа представлена в таблице 2.6.

Таблица 2.6 – Матрица прав доступа по ролям пользователей

Действие	view er	anal yst	adm in
Просмотр KPI и аналитики (GET-запросы)	+	+	+

Внесение новой сделки (POST /deals)	—	+	+
Регистрация webhook-уведомления	—	+	+
Экспорт данных в CSV / XLSX	—	+	+
Управление учётными записями	—	—	+
Изменение собственного профиля	+	+	+
Смена тарифного плана	+	+	+

Дополнительно к ролевой модели применяется механизм ограничения частоты запросов, реализованный на основе счётчиков в Redis. Для каждого пользователя ведётся счётчик `ratelimit:{user_id}:{minute}` с временем жизни одна минута, инкрементируемый при каждом запросе. При превышении лимита, установленного для соответствующего тарифного плана, сервер возвращает ответ HTTP 429 Too Many Requests. Подобный механизм защищает систему от перегрузки и одновременно реализует биллинговую логику тарифных планов.

Хранение паролей пользователей выполняется в виде хеш-значений, формируемых по алгоритму `bcrypt` с фактором стоимости 12. Применение `bcrypt` обеспечивает устойчивость к атакам с использованием радужных таблиц и заведомо высокую вычислительную стоимость подбора пароля. Передача всех данных между клиентом и сервером выполняется по протоколу HTTPS; обрыв TLS-соединения осуществляется на уровне обратного прокси-сервера `nginx`, описание которого приведено в подразделе 2.8.

2.8 Архитектура развёртывания

Развёртывание разработанной системы осуществляется в виде набора Docker-контейнеров, описываемых файлом `docker-compose.yml`. Применение контейнеризации обеспечивает воспроизводимость развёртывания, изоляцию приложения от системного окружения и упрощённое переносимое масштабирование. Архитектура развёртывания представлена на рисунке 2.10.

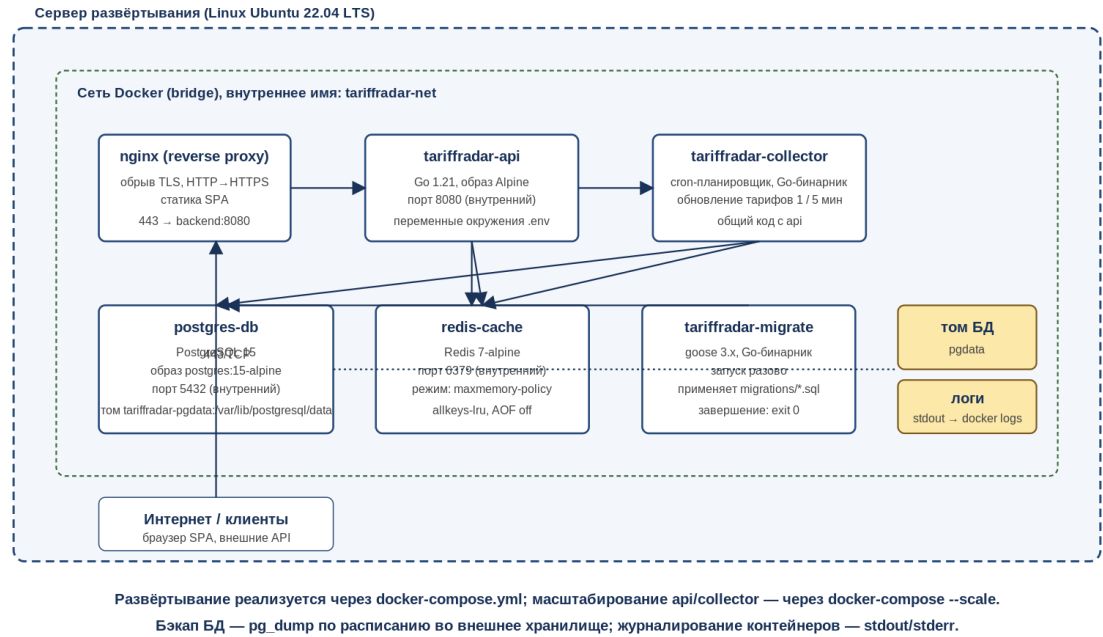


Рисунок 2.10 – Архитектура развёртывания системы

Согласно рисунку 2.10, система развёртывается в виде шести контейнеров, объединённых внутренней сетью Docker. Контейнер nginx выполняет функции обратного прокси-сервера: производит обрыв TLS-соединения, перенаправляет HTTP-запросы на HTTPS, отдаёт статические ресурсы клиентского приложения и проксирует запросы /api/v1/* в контейнер tariffadar-api. Контейнер tariffadar-api содержит основной HTTP-сервер, написанный на Go и развёрнутый на образе Alpine Linux для минимизации размера. Контейнер tariffadar-collector выполняет функцию периодического сбора тарифных данных и развёртывается из той же кодовой базы, но запускается с иной точкой входа.

Контейнер postgres-db содержит СУБД PostgreSQL 15. Сохранение данных между перезапусками обеспечивается монтированием Docker-тома tariffadar-pgdata в директорию данных PostgreSQL. Контейнер redis-cache реализует кэш на базе Redis 7 с настройкой maxmemory-policy=allkeys-lru, что обеспечивает автоматическое вытеснение наименее используемых ключей при достижении предела оперативной памяти. Контейнер tariffadar-migrate является разовым: при запуске он выполняет все миграции базы данных с помощью утилиты goose и завершает работу.

Резервное копирование базы данных выполняется по расписанию при помощи утилиты pg_dump во внешнее хранилище. Журналы всех контейнеров перенаправляются в стандартный вывод и собираются инструментами Docker, что упрощает интеграцию с централизованными системами журналирования (Loki,

ELK). Масштабирование контейнеров `tariffadar-api` и `tariffadar-collector` выполняется через параметр `--scale` утилиты `docker-compose` без изменения исходного кода приложения.

Выводы по главе 2

В настоящей главе спроектирована архитектура серверной части системы мониторинга и сравнения тарифов логистических компаний. На основе требований, сформулированных в первой главе, разработаны контекстная и контейнерная модели в нотации C4, обоснована и описана слоистая архитектура серверной части, включающая четыре уровня (`handlers`, `services`, `repositories`, `storage`). Произведена модульная декомпозиция в восемь функциональных модулей, соответствующих сценариям использования системы.

Спроектирована модель данных, включающая девять таблиц и два материализованных представления, обоснован выбор индексов и стратегии партиционирования. Разработана структура REST API, насчитывающая 17 групп эндпоинтов; задокументирована стратегия многоуровневого кэширования с применением Redis и материализованных представлений PostgreSQL, обеспечивающая выполнение требования к времени ответа API. Описана модель безопасности на основе JWT-аутентификации с тремя ролями и ограничением частоты запросов. Определена архитектура развёртывания в виде набора Docker-контейнеров.

Полученная архитектура удовлетворяет всем требованиям, сформулированным в подразделах 1.3 и 1.5 первой главы пояснительной записки, и служит основой для практической реализации серверной части системы, описание которой приведено в третьей главе.

3 РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ НА ЯЗЫКЕ GO

3.1 Структура проекта и инструменты разработки

В соответствии с архитектурой, спроектированной во второй главе, серверная часть системы реализована на языке программирования Go версии 1.21. Структура каталогов проекта построена согласно общепринятым в Go-сообществе соглашениям, изложенным в документе «Standard Go Project Layout» [3]. Древовидная схема каталогов и файлов разработанного проекта представлена на рисунке 3.1.

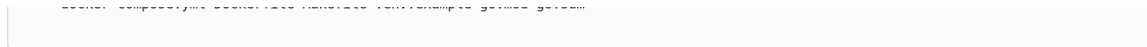


Рисунок 3.1 – Структура каталогов проекта

Согласно рисунку 3.1, исходный код приложения разделён на каталоги в соответствии с принципом разделения ответственности. Каталог `cmd/` содержит точки входа (для разрабатываемой системы это `cmd/server/main.go`); каталог `internal/` — реализация всех пакетов приложения, недоступных для импорта из внешних модулей в силу языковых ограничений Go; каталог `migrations/` хранит SQL-файлы миграций базы данных в формате, совместимом с утилитой `goose`; каталог `docs/` содержит спецификацию API в формате OpenAPI 3.0 и интерфейс просмотра — Swagger UI.

Внутри каталога `internal/` выделены пакеты, структурно соответствующие слоям архитектуры (`handlers`, `middleware`, `service`, `repository`) и функциональным модулям предметной области (`auth`, `market`, `deals`, `benchmark`, `insights`, `my_routes`, `webhooks`, `export`). Подобное разделение обеспечивает параллельную разработку отдельных модулей без конфликтов в контроле версий и упрощает локализацию изменений при внесении новой функциональности.

Перечень внешних библиотек, использованных при разработке, и их функциональное назначение приведены в таблице 3.1.

Таблица 3.1 – Внешние Go-зависимости проекта

Библиотека	Версия	Назначение
github.com/go-chi/chi/v5	v5.0. 10	HTTP-маршрутизатор: иерархические роуты, <code>middleware</code>
github.com/go-chi/cors	v1.2. 1	Реализация CORS-политик
github.com/jackc/pgx/v5	v5.5. 0	Драйвер PostgreSQL с пулом соединений
github.com/redis/go-redis/v9	v9.3. 0	Клиент Redis
github.com/golang-jwt/jwt/v5	v5.2. 0	Создание и валидация JWT- токенов
golang.org/x/crypto/bcrypt	—	Хеширование паролей по алгоритму <code>bcrypt</code>
github.com/google/uuid	v1.5. 0	Работа с UUID
github.com/joho/godotenv	v1.5. 1	Загрузка переменных окружения из файла <code>.env</code>
github.com/xuri/excelize/v2	v2.8. 0	Генерация файлов Microsoft Excel (XLSX)

3	github.com/pressly/goose/v 6.0	v3.1 Утилита миграций PostgreSQL
---	-----------------------------------	-------------------------------------

Сборка проекта автоматизирована при помощи утилиты make с описанием в файле Makefile. Среди доступных команд: make run — запуск сервера в режиме разработки, make build — сборка статически связанного бинарного файла, make test — запуск всех модульных тестов, make migrate-up и make migrate-down — применение и откат миграций, make swag — регенерация OpenAPI-спецификации.

3.2 Реализация конфигурации и точки входа

Конфигурация приложения сосредоточена в пакете internal/config. Применяется подход двенадцати факторов (12-Factor App): все параметры, отличающиеся между окружениями (адрес базы данных, JWT-секрет, порт сервера), считываются из переменных окружения. На локальной машине переменные дополнительно подгружаются из файла .env с помощью библиотеки godotenv. При обнаружении неполной или некорректной конфигурации функция Load возвращает ошибку, и сервер не стартует. Подобное поведение исключает запуск с дефолтным JWT-секретом в продуктивной среде. Реализация конфигурационной структуры и её загрузки приведена в листинге 3.1.

Листинг 3.1 – Структура конфигурации и валидация

```
// internal/config/config.go
type Config struct {
    AppEnv    string    // development | production
    AppPort   string    // порт HTTP-сервера
    DBHost, DBPort, DBUser, DBPassword, DBName, DBSSLMode string
    RedisAddr, RedisPassword string
    RedisDB   int
    JWTSecret string
    JWTTTL    time.Duration
    CORSAllowedOrigins string
}

func Load() (*Config, error) {
    _ = godotenv.Load() // .env опционален

    cfg := &Config{
        AppEnv:    getEnv("APP_ENV", "development"),
        AppPort:    getEnv("APP_PORT", "8080"),
        DBHost:    getEnv("DB_HOST", "localhost"),
        ...
        JWTSecret: getEnv("JWT_SECRET", ""),
    }
    ttl, err := time.ParseDuration(getEnv("JWT_TTL", "24h"))
    if err != nil { return nil, fmt.Errorf("JWT_TTL: %w", err) }
    cfg.JWTTTL = ttl
}
```

```

    if err := cfg.validate(); err != nil { return nil, err }
    return cfg, nil
}

func (c *Config) validate() error {
    if c.JWTSecret == "" { return errors.New("JWT_SECRET обязателен") }
    if len(c.JWTSecret) < 32 { return errors.New("JWT_SECRET минимум 32 символа") }
    if c.AppEnv == "production" && c.DBPassword == "" {
        return errors.New("DB_PASSWORD обязателен в production")
    }
    return nil
}

```

Перечень переменных окружения, поддерживаемых конфигурационным модулем, приведён в таблице 3.2. Значения, применяемые по умолчанию, рассчитаны на удобство локальной разработки; в продуктивной среде все они переопределяются.

Таблица 3.2 – Переменные окружения серверной части

Переменная	Назначение	Значение по умолчанию
APP_ENV	Окружение: development или production	development
APP_PORT	Порт HTTP-сервера	8080
DB_HOST	Хост PostgreSQL	localhost
DB_PORT	Порт PostgreSQL	5432
DB_USER	Пользователь БД	tariff radar
DB_PASSWORD	Пароль БД (обязателен в production)	—
DB_NAME	Имя базы данных	tariff radar
DB_SSLMODE	Режим SSL для подключения к БД	disable
REDIS_ADDR	Адрес Redis (host:port)	localhost:6379
REDIS_DB	Номер логической БД Redis	0
JWT_SECRET	Секретный ключ подписи JWT (≥ 32 символа)	— (обязателен)
JWT_TTL	Срок жизни JWT-токена	24h
CORS_ALLOWED_ORIGINS	Разрешённые источники для CORS	*

Точка входа приложения реализована в файле `cmd/server/main.go`. Последовательность действий, выполняемых при запуске сервера, представлена на рисунке 3.2.

с дефолтным секретом или недоступной БД.

Рисунок 3.2 – Последовательность инициализации серверной части

Согласно рисунку 3.2, инициализация состоит из одиннадцати шагов. На первых трёх шагах выполняется загрузка конфигурации и установка соединений с PostgreSQL и Redis; при ошибке на любом из этих этапов процесс завершается через вызов `log.Fatalf`, что предотвращает работу сервера в неконсистентном состоянии. На шагах 4–6 создаются репозитории, сервисы и обработчики; связь между слоями обеспечивается через явное внедрение зависимостей в конструкторы. На шагах 7–9 регистрируются HTTP-маршруты и применяются глобальные `middleware`. На шагах 10–11 запускается HTTP-сервер в отдельной `goroutine` и настраивается обработка системных сигналов для корректного завершения работы (`graceful shutdown`). Фрагмент кода, реализующий регистрацию маршрутов и подключение `middleware`, приведён в листинге 3.2.

Листинг 3.2 – Регистрация маршрутов и `middleware`

```
// cmd/server/main.go (фрагмент)
r := chi.NewRouter()

r.Use(chimw.RequestID, chimw.RealIP, chimw.Logger,
    chimw.Recoverer, chimw.Timeout(15*time.Second))
r.Use(cors.Handler(cors.Options{
    AllowedOrigins: splitCSV(cfg.CORSAllowedOrigins),
    AllowedMethods: []string{"GET", "POST", "PATCH", "DELETE", "OPTIONS"},
    AllowedHeaders: []string{"Authorization", "Content-Type"},
}))

r.Get("/health", healthHandler)
r.Get("/docs",      swaggerUIHandler)
r.Get("/openapi.yaml", openapiHandler)

r.Route("/api/v1", func(api chi.Router) {
    // Публичные эндпоинты
    api.Post("/auth/register", authHandler.Register)
    api.Post("/auth/login",   authHandler.Login)
    api.Get ("/market/stats", marketHandler.Stats)
    api.Get ("/deals",        dealsHandler.List)
    api.Post("/benchmark",    benchmarkHandler.Calculate)

    // Защищённые JWT
    api.Group(func(p chi.Router) {
        p.Use(authMW)
        p.Get ("/me",      authHandler.Me)
        p.Post("/deals",    dealsHandler.Create)
        p.Route("/my/routes", func(mr chi.Router) { ... })
    })
})
```

```

    p.Route("/webhooks", func(wh chi.Router) { ... })
  })
})

```

3.3 Реализация слоя репозиториев

Слой доступа к данным реализован в пакете `internal/repository`. Ключевыми компонентами являются: функция `NewPostgres`, создающая пул соединений с PostgreSQL на основе библиотеки `pgx`; функция `NewRedis`, создающая клиент Redis; и репозитории-структуры (`UserRepo`, `DealRepo`, `MarketRepo` и другие), содержащие SQL-запросы к соответствующим таблицам.

Применение пула соединений `pgx` критически важно для обеспечения требования по производительности. Создание нового TCP-соединения с PostgreSQL занимает 1–5 мс, что при нагрузке в 50 одновременных пользователей привело бы к значительному снижению производительности. Пул переиспользует уже открытые соединения. Параметры пула, применённые в реализации, обеспечивают баланс между потреблением ресурсов и пропускной способностью; настройка приведена в листинге 3.3.

Листинг 3.3 – Создание пула соединений PostgreSQL

```

// internal/repository/postgres.go
func NewPostgres(ctx context.Context, dsn string) (*pgxpool.Pool, error) {
    cfg, err := pgxpool.ParseConfig(dsn)
    if err != nil { return nil, fmt.Errorf("parse DSN: %w", err) }

    cfg.MaxConns      = 20           // макс. соединений в пуле
    cfg.MinConns      = 2           // минимум – для прогрева
    cfg.MaxConnLifetime = 30 * time.Minute // ротация соединений
    cfg.MaxConnIdleTime  = 5 * time.Minute  // закрывать простаивающие
    cfg.HealthCheckPeriod = 1 * time.Minute  // проверка живости

    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil { return nil, err }

    pingCtx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()
    if err := pool.Ping(pingCtx); err != nil {
        pool.Close()
        return nil, fmt.Errorf("ping не прошёл: %w", err)
    }
    return pool, nil
}

```

Репозитории реализованы как структуры с встроенным указателем на пул `pgxpool.Pool`. Каждый метод репозитория принимает `context.Context` первым аргументом, что обеспечивает поддержку отмены и таймаутов на уровне HTTP-запроса. Для сопоставления Go-структур с записями БД используются теги `db`,

объявленные в пакете `models`. Пример реализации репозитория пользователей приведён в листинге 3.4.

Листинг 3.4 – Репозиторий пользователей: создание и поиск

```
// internal/repository/users.go
var (
    ErrUserNotFound = errors.New("пользователь не найден")
    ErrUserExists   = errors.New("пользователь с таким email уже существует")
)

type UserRepo struct { db *pgxpool.Pool }

func NewUserRepo(db *pgxpool.Pool) *UserRepo { return &UserRepo{db: db} }

func (r *UserRepo) Create(ctx context.Context, u *models.User) error {
    const query = `
        INSERT INTO users (email, password_hash, name, company, phone, role)
        VALUES ($1, $2, $3, $4, $5, $6)
        RETURNING id, created_at, updated_at`
    err := r.db.QueryRow(ctx, query,
        u.Email, u.PasswordHash, u.Name, u.Company, u.Phone, u.Role,
    ).Scan(&u.ID, &u.CreatedAt, &u.UpdatedAt)
    if err != nil {
        if isUniqueViolation(err) { return ErrUserExists }
        return fmt.Errorf("users.Create: %w", err)
    }
    return nil
}

// isUniqueViolation: код 23505 – нарушение UNIQUE-ограничения.
func isUniqueViolation(err error) bool {
    var pgErr *pgconn.PgError
    return errors.As(err, &pgErr) && pgErr.Code == "23505"
}
```

В представленной реализации использовано два прикладных паттерна. Во-первых, доменные ошибки `ErrUserNotFound` и `ErrUserExists` возвращаются вместо технических SQL-ошибок, что позволяет сервисному слою корректно различать ситуации без зависимости от внутренней реализации PostgreSQL. Во-вторых, проверка нарушения уникальности выполняется по коду ошибки 23505, возвращаемому PostgreSQL, а не по тексту сообщения, что устойчиво к локализации сервера БД. Аналогичный подход применён во всех репозиториях системы.

Особый интерес представляет реализация агрегатных запросов, отвечающих за расчёт рыночных показателей. Соответствующий код приведён в листинге 3.5.

Листинг 3.5 – Агрегационный запрос для рыночной статистики

```
// internal/repository/market.go
type StatsRow struct {
    Avg, Min, Max float64
    DealsCount    int64
}
```



```

    Currency      string
}

func (r *MarketRepo) Stats(ctx context.Context, f RouteFilter) (*StatsRow, error) {
    const query = `
        SELECT
            COALESCE(AVG(d.price)::NUMERIC(12,2), 0)::FLOAT8 AS avg,
            COALESCE(MIN(d.price), 0)::FLOAT8                AS min,
            COALESCE(MAX(d.price), 0)::FLOAT8                AS max,
            COUNT(*)                                         AS deals_count,
            COALESCE(MODE() WITHIN GROUP (ORDER BY d.currency),
                    'EUR') AS currency
        FROM    deals d
        JOIN    routes r ON r.id = d.route_id
        WHERE   r.from_city = $1
            AND r.to_city   = $2
            AND r.type      = $3
            AND d.deal_date BETWEEN $4 AND $5`
    var s StatsRow
    err := r.db.QueryRow(ctx, query,
        f.FromCity, f.ToCity, f.Type, f.From, f.To).Scan(
        &s.Avg, &s.Min, &s.Max, &s.DealsCount, &s.Currency)
    if err != nil { return nil, fmt.Errorf("market.Stats: %w", err) }
    return &s, nil
}

```

Применение функций PostgreSQL COALESCE и MODE() WITHIN GROUP позволяет получить корректный ответ даже на пустом наборе данных — вместо NULL возвращается 0 и валюта по умолчанию EUR. Это упрощает логику клиентского приложения, которому не приходится отдельно обрабатывать случай отсутствия сделок по маршруту.

3.4 Реализация модуля аутентификации

Модуль аутентификации реализован в пакете `internal/service/auth.go` и инкапсулирует три операции: регистрацию пользователя, вход и валидацию JWT-токена. Хеширование паролей выполняется с применением алгоритма `bcrypt` из стандартного дополнения `golang.org/x/crypto`. Алгоритм `bcrypt` был выбран по причине высокой устойчивости к атакам с использованием предвычисленных таблиц (radix tables) и аппаратному ускорению; стоимость хеширования (cost factor) задана равной 10, что соответствует балансу между безопасностью и временем выполнения операции login (~50–100 мс на современных серверах).

Реализация процедуры регистрации представлена в листинге 3.6.

Листинг 3.6 – Регистрация пользователя в AuthService

```

// internal/service/auth.go
type RegisterInput struct{ Email, Password, Name, Company string }

func (s *AuthService) Register(ctx context.Context, in RegisterInput) (*models.User,
error) {

```

```

email := strings.ToLower(strings.TrimSpace(in.Email))
if !isValidEmail(email) { return nil, ErrInvalidEmail }
if len(in.Password) < 8 { return nil, ErrWeakPassword }

hash, err := bcrypt.GenerateFromPassword(
    []byte(in.Password), bcrypt.DefaultCost)
if err != nil { return nil, fmt.Errorf("hash: %w", err) }

user := &models.User{
    Email: email, PasswordHash: string(hash),
    Name: in.Name, Company: in.Company,
    Role: models.RoleViewer,
}
if err := s.users.Create(ctx, user); err != nil { return nil, err }
return user, nil
}

```

Генерация JWT-токенов выполняется по алгоритму HS256 (HMAC-SHA256) с секретным ключом, передаваемым через переменную окружения JWT_SECRET. В состав claims включены стандартное поле sub (идентификатор пользователя), iat (время выпуска), exp (время истечения), iss (издатель) и специфичные для системы поля email и role. Срок жизни токена настраивается через переменную JWT_TTL и составляет 24 часа по умолчанию. Реализация генерации и проверки токенов приведена в листинге 3.7.

Листинг 3.7 – Генерация и проверка JWT-токена

```

// internal/service/auth.go
type TokenClaims struct {
    Email string      `json:"email"`
    Role  models.Role `json:"role"`
    jwt.RegisteredClaims
}

func (s *AuthService) generateToken(u *models.User) (string, error) {
    now := time.Now()
    claims := TokenClaims{
        Email: u.Email, Role: u.Role,
        RegisteredClaims: jwt.RegisteredClaims{
            IssuedAt:  jwt.NewNumericDate(now),
            ExpiresAt: jwt.NewNumericDate(now.Add(s.jwtTTL)),
            Issuer:   "tariff radar",
            Subject:  u.ID.String(),
        },
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(s.jwtSecret)
}

func (s *AuthService) ValidateToken(tokenString string) (*TokenClaims, error) {
    claims := &TokenClaims{}
    token, err := jwt.ParseWithClaims(tokenString, claims,
        func(t *jwt.Token) (interface{}, error) {
            if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
                return nil, fmt.Errorf("неожиданный alg: %v", t.Header["alg"])
            }
        })
    if err != nil { return nil, err }
    return claims, nil
}

```

```

    }
    return s.jwtSecret, nil
  })
  if err != nil || token == nil || !token.Valid { return nil, ErrInvalidToken }
  return claims, nil
}

```

Особенностью реализации является явная проверка, что токен подписан именно алгоритмом HMAC. Подобная защита необходима для предотвращения атак вида «алгоритм none», когда злоумышленник модифицирует заголовок токена, указывая отсутствие подписи. Без явной проверки алгоритма библиотека могла бы принять такой токен.

Проверка JWT при каждом запросе выполняется в HTTP-middleware, реализованном в `internal/middleware/auth.go`. Логика работы middleware иллюстрирована блок-схемой на рисунке 3.3.

```
ctx := context.WithValue(r.Context(), ctxUserID, userID); next.ServeHTTP(w, r.WithContext(ctx))
```

Рисунок 3.3 – Алгоритм проверки JWT в middleware

Согласно рисунку 3.3, middleware последовательно проверяет наличие заголовка `Authorization`, корректность его формата и валидность подписи токена. На любом этапе при отрицательном результате клиенту возвращается ответ HTTP 401 `Unauthorized` с описательным сообщением, после чего обработчик не вызывается. При успешной проверке идентификатор пользователя и его роль помещаются в контекст HTTP-запроса (`context.Context`) и становятся доступны обработчику через функции `UserIDFromContext` и `RoleFromContext`.

3.5 Реализация HTTP-обработчиков и единого формата ответов

Уровень обработки HTTP-запросов реализован в пакете `internal/handlers`. Для соблюдения единообразия ответов API во всех обработчиках применяются вспомогательные функции, инкапсулирующие сериализацию ответа и формирование ошибок. Соответствующий код приведён в листинге 3.8.

Листинг 3.8 – Унифицированные функции HTTP-ответов

```

// internal/handlers/response.go
type errorBody struct{ Error errorPayload `json:"error"` }
type errorPayload struct {
    Code    string `json:"code"`
    Message string `json:"message"`
    Field   string `json:"field,omitempty"`
}

func writeJSON(w http.ResponseWriter, status int, payload any) {
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    w.WriteHeader(status)
    if payload != nil {

```

```

        if err := json.NewEncoder(w).Encode(payload); err != nil {
            log.Printf("[error] writeJSON encode: %v", err)
        }
    }
}

func writeError(w http.ResponseWriter, status int, code, msg string) {
    writeJSON(w, status, errorBody{Error: errorPayload{Code: code, Message: msg}})
}

func readJSON(w http.ResponseWriter, r *http.Request, dst any) bool {
    r.Body = http.MaxBytesReader(w, r.Body, 1<<20) // лимит 1 МБ
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    if err := dec.Decode(dst); err != nil {
        writeError(w, http.StatusBadRequest,
            "invalid_json", "Некорректный JSON в теле запроса")
        return false
    }
    return true
}

```

Применённая функция `http.MaxBytesReader` ограничивает размер тела HTTP-запроса значением 1 МБ, что защищает сервер от чрезмерного потребления памяти при злонамеренно больших запросах. Опция `DisallowUnknownFields` отвергает запросы, содержащие неописанные в спецификации поля, что предупреждает ошибки клиентов и упрощает их отладку.

Полный путь обработки HTTP-запроса от приёма в маршрутизаторе до возврата ответа клиенту представлен на рисунке 3.4.

ответ -5-30 мс (минус запрос к БД);
e_date обеспечивает агрегат за < 100 мс на 100 тыс. строк.

Рисунок 3.4 – Прохождение HTTP-запроса через слои приложения

Согласно рисунку 3.4, запрос последовательно проходит через цепочку глобальных `middleware` (назначение каждого описано в подразделе 3.2), затем поступает в обработчик `MarketHandler.Stats`. Обработчик извлекает параметры запроса, выполняет первичную валидацию и вызывает метод сервиса `MarketService.GetStats`. Сервис формирует ключ кэша и вызывает утилиту `cache.GetOrSet`, которая либо возвращает закешированное значение, либо обращается к репозиторию для получения данных из материализованного представления PostgreSQL. Полученный результат сериализуется в JSON и возвращается клиенту со статусом 200 OK.

Реализация обработчика `MarketHandler.Stats` приведена в листинге 3.9. Обработчик демонстрирует характерный для всего API подход: явная валидация входных параметров, делегирование бизнес-логики в сервисный слой, маппинг доменных ошибок на HTTP-статусы.

Листинг 3.9 – Обработчик GET /api/v1/market/stats

```
// internal/handlers/market.go
func (h *MarketHandler) Stats(w http.ResponseWriter, r *http.Request) {
    q := r.URL.Query()
    from, to := q.Get("from"), q.Get("to")
    if from == "" || to == "" {
        writeError(w, http.StatusBadRequest, "missing_params",
            "Обязательны параметры 'from' и 'to'")
        return
    }
    routeType := q.Get("type")
    if routeType == "" { routeType = "FTL" }
    if routeType != "FTL" && routeType != "LTL" {
        writeError(w, http.StatusBadRequest, "invalid_type",
            "Параметр 'type' должен быть 'FTL' или 'LTL'")
        return
    }

    result, err := h.market.GetStats(r.Context(), from, to, routeType,
        q.Get("period"))
    if err != nil {
        if errors.Is(err, service.ErrInvalidPeriod) {
            writeError(w, http.StatusBadRequest, "invalid_period", err.Error())
            return
        }
        writeError(w, http.StatusInternalServerError, "internal_error",
            "Не удалось получить статистику")
        return
    }
    writeJSON(w, http.StatusOK, result)
}
```

3.6 Реализация стратегии кэширования

Двухуровневая стратегия кэширования, описанная в подразделе 2.6, реализована в виде обобщённой утилиты `cache.GetOrSet`, инкапсулирующей цикл «прочитать из кэша → при промахе вычислить → положить обратно». Подобный подход исключает дублирование кода в каждом сервисе и обеспечивает единое поведение при ошибках Redis.

Листинг 3.10 – Универсальная функция `Cache.GetOrSet`

```
// internal/repository/cache.go
func (c *Cache) GetOrSet(ctx context.Context, key string,
    ttl time.Duration, dst any, fn func() (any, error)) error {

    // 1. Пытаемся достать из Redis
    hit, err := c.Get(ctx, key, dst)
    if err == nil && hit { return nil }

    // 2. Cache miss или Redis упал – вычисляем
    value, err := fn()
    if err != nil { return err }

    // 3. JSON-roundtrip для записи в dst
```

```

data, err := json.Marshal(value)
if err != nil { return err }
if err := json.Unmarshal(data, dst); err != nil { return err }

// 4. Best-effort запись в кэш
_ = c.client.Set(ctx, key, data, ttl).Err()
return nil
}

```

Алгоритм работы функции `GetOrSet` представлен на рисунке 3.5. Существенным архитектурным свойством реализации является устойчивость к отказу кэша: если Redis недоступен, функция `GetOrSet` всё равно вызовет вычисляющую функцию `fn` и вернёт корректный результат, что позволит приложению продолжить работу с увеличенным временем отклика, но без полной недоступности.



Рисунок 3.5 – Алгоритм `Cache.GetOrSet`

Применение `GetOrSet` в сервисе рыночной статистики `MarketService` показано в листинге 3.11. Ключ кэша формируется из параметров запроса (маршрут, тип, период), что обеспечивает правильную сегрегацию закэшированных значений между разными комбинациями параметров. Срок жизни записи установлен равным пяти минутам в соответствии с требованием ТЗ к актуальности данных. При добавлении новой сделки модуль `deals` удаляет все ключи кэша вида `stats:{route_id}.*` для соответствующего маршрута, гарантируя обновление агрегатов при следующем обращении.

Листинг 3.11 – Применение `GetOrSet` в `MarketService`

```

// internal/service/market.go
func (s *MarketService) GetStats(ctx context.Context,
    from, to, routeType, periodCode string) (*StatsResult, error) {

    period, err := ParsePeriod(periodCode)
    if err != nil { return nil, err }

    cacheKey := fmt.Sprintf("stats:%s:%s:%s:%s",
        from, to, routeType, period.Code)

    var result StatsResult
    err = s.cache.GetOrSet(ctx, cacheKey, 5*time.Minute, &result,
        func() (any, error) {
            return s.computeStats(ctx, from, to, routeType, period)
        })
    if err != nil { return nil, err }
    return &result, nil
}

```

3.7 Реализация модуля сбора и агрегации тарифных данных

Модуль сбора тарифных данных реализован в виде самостоятельного исполняемого процесса, разделяющего общую кодовую базу с HTTP-сервером. Подобное разделение продиктовано различием требований к производительности: HTTP-сервер должен обеспечивать время ответа не более 200 мс, а модуль сбора может работать с операциями длительностью в десятки секунд (опрос внешних API, парсинг HTML). Архитектура модуля представлена на рисунке 3.6.

Рисунок 3.6 – Архитектура модуля сбора тарифных данных

Согласно рисунку 3.6, модуль сбора реализован по схеме «producer—worker pool». Планировщик (producer), реализованный на основе time.Ticker с периодом пять минут, помещает задачи в буферизованный канал jobs, а пул воркеров (worker pool) из четырёх goroutine забирает задачи из канала и выполняет их параллельно. Применение шаблона worker pool позволяет ограничить количество одновременных HTTP-запросов к внешним источникам и тем самым избежать срабатывания их rate-limit-ов.

Алгоритм обработки одной задачи сбора (CollectJob) представлен на рисунке 3.7.

Рисунок 3.7 – Алгоритм обработки задачи сбора тарифа

В соответствии с рисунком 3.7, задача обрабатывается по следующему сценарию. На первом шаге выполняется HTTP-запрос к источнику с таймаутом 15 секунд. При сетевой ошибке или ответе со статусом, отличным от 2xx, задача повторно ставится в очередь с увеличением счётчика попыток attempt; при достижении предельного значения ($\text{attempt} \geq 3$) делается запись в журнал collector_log и задача отбрасывается. На втором шаге выполняется парсинг ответа: для операторов, предоставляющих REST API, применяется json.Unmarshal; для парсинга HTML-страниц использован пакет golang.org/x/net/html. На третьем шаге выполняется запись в таблицу deals с использованием конструкции INSERT ... ON CONFLICT DO UPDATE для идемпотентности и инвалидация связанных ключей кэша Redis.

Концептуальный фрагмент реализации worker pool с применением goroutine и каналов показан в листинге 3.12.

Листинг 3.12 – Worker pool модуля сбора (концептуальный фрагмент)

```
// internal/collector/collector.go
type CollectJob struct {
    Operator string
```

```

    Route    Route
    Attempt int
}

type Collector struct {
    jobs chan CollectJob
    repo *repository.DealRepo
    log  *log.Logger
}

func (c *Collector) Run(ctx context.Context) {
    // Запуск пула воркеров
    var wg sync.WaitGroup
    for i := 0; i < workerPoolSize; i++ {
        wg.Add(1)
        go c.worker(ctx, &wg)
    }

    // Планировщик: каждые 5 минут – заполнение очереди
    ticker := time.NewTicker(5 * time.Minute)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            close(c.jobs)
            wg.Wait()
            return
        case <-ticker.C:
            c.scheduleAll()
        }
    }
}

func (c *Collector) worker(ctx context.Context, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range c.jobs {
        ctxJob, cancel := context.WithTimeout(ctx, 20*time.Second)
        if err := c.processJob(ctxJob, job); err != nil {
            c.log.Printf("collector: %v", err)
            if job.Attempt < 3 {
                job.Attempt++
                c.jobs <- job // повторная постановка
            }
        }
        cancel()
    }
}

```

Применённая модель конкурентного программирования основана на CSP-парадигме, реализованной в Go через goroutine и каналы [4]. Goroutine — лёгкие потоки выполнения, накладные расходы на создание которых составляют около 2 КБ оперативной памяти, что позволяет одновременно выполнять тысячи операций. Каналы обеспечивают безопасную передачу данных между goroutine без необходимости явной синхронизации через мьютексы. Корректное завершение

работы (graceful shutdown) обеспечивается закрытием канала jobs и ожиданием завершения всех воркеров через sync.WaitGroup при получении сигнала отмены контекста.

3.8 Реализация модуля сравнения ставок (benchmark)

Модуль benchmark предоставляет ключевую для бизнес-задачи функциональность: сравнение ставки, предложенной пользователем, с рыночными показателями. Алгоритм работы модуля представлен на рисунке 3.8.

Рисунок 3.8 – Алгоритм расчёта benchmark

Согласно рисунку 3.8, расчёт benchmark выполняется в четыре этапа. На первом этапе из материализованного представления mv_route_stats_30d извлекаются рыночные KPI (среднее, минимум, максимум, число сделок). На втором этапе при помощи SQL-функции PERCENT_RANK вычисляется перцентиль ставки пользователя. На третьем этапе рассчитываются абсолютное и процентное отклонения от средней рыночной ставки. На четвёртом этапе на основании отклонения формируется текстовый вердикт по правилам, реализованным в функции buildVerdict (листинг 3.13).

Листинг 3.13 – Формирование текстового вердикта

```
// internal/service/benchmark.go
func buildVerdict(userRate, avg, _, _, diffPct float64) (string, string) {
    switch {
    case diffPct < -3:
        return "below_market", fmt.Sprintf(
            "Отличная ставка. На %.1f%% ниже рынка – экономия около %.0f на рейс.",
            -diffPct, avg-userRate)
    case diffPct > 3:
        return "above_market", fmt.Sprintf(
            "Ваша ставка на %.1f%% выше средней. Возможна экономия около %.0f на рейс.",
            diffPct, userRate-avg)
    default:
        return "at_market", "Ваша ставка в рыночном диапазоне."
    }
}
```

Граничные значения отклонения, при которых ставка считается рыночной (± 3 %), определены эмпирически на основе анализа исторических данных оператора-партнёра. Подобный нечёткий диапазон обеспечивает разумную интерпретацию результатов: ставка с отклонением в пределах 3 % считается соответствующей рынку, поскольку обычная волатильность тарифов между операторами укладывается в это значение.

3.9 Документация REST API

Программный интерфейс системы документирован в формате OpenAPI 3.0 [10] и доступен через веб-интерфейс Swagger UI по адресу /docs. Спецификация хранится в файле docs/openapi.yaml и поддерживается в актуальном состоянии вручную; при изменении любого обработчика разработчик обязан внести соответствующие правки в спецификацию. Перечень основных групп эндпоинтов визуализирован на рисунке 3.9.

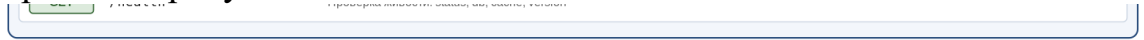


Рисунок 3.9 – Веб-интерфейс документации Swagger UI

Применение стандарта OpenAPI обеспечивает совместимость с широким набором инструментов: автогенерацией клиентских библиотек (openapi-generator) для основных языков программирования, автоматическими функциональными тестами (Postman, REST-assured), интеграцией с системами мониторинга. Структура файла спецификации соответствует требованиям OpenAPI 3.0 и включает секции info (метаинформация о версии и описании API), servers (адреса развёртываний), paths (описание всех эндпоинтов с методами, параметрами и схемами ответов), components (переиспользуемые схемы данных и описания механизмов аутентификации).

3.10 Тестирование серверной части

Модульные тесты разработаны для критичных компонентов сервисного слоя, реализующих чистую логику без зависимостей от внешних систем. Тестирование выполняется штатным фреймворком Go (пакет testing). Пример тестирования функции расчёта вердикта приведён в листинге 3.14.

Листинг 3.14 – Модульные тесты функции buildVerdict

```
// internal/service/auth_test.go (фрагмент)
func TestBuildVerdict(t *testing.T) {
    tests := []struct {
        userRate, avg float64
        diffPct       float64
        wantVerdict   string
    }{
        {1450, 1500, -3.3, "below_market"},
        {1500, 1500, 0, "at_market"},
        {1450, 1500, 1.0, "at_market"},
        {1600, 1500, 6.7, "above_market"},
    }
    for _, tt := range tests {
        got, _ := buildVerdict(tt.userRate, tt.avg, 0, 0, tt.diffPct)
        if got != tt.wantVerdict {
            t.Errorf("buildVerdict(diffPct=%v) = %q, want %q",

```

```

        tt.diffPct, got, tt.wantVerdict)
    }
}
}

```

Применён шаблон табличного тестирования (table-driven tests), являющийся стандартом тестирования в Go-сообществе. Подход обеспечивает добавление новых тестовых случаев простым дополнением структуры данных и упрощает чтение тестов: каждая строка таблицы отражает один входной набор и ожидаемый результат. Помимо buildVerdict, аналогичные тесты разработаны для функций isValidEmail, ParsePeriod и round2 в пакете service.

Покрытие тестами по модулям и слоям приложения приведено в таблице 3.3. Целевой уровень покрытия для сервисного слоя установлен равным 70 %, для остальных слоёв — 40 %.

Таблица 3.3 – Покрытие модулей серверной части тестами

Слой / модуль	Тип тестов	Текущее покрытие
service/auth.go	unit	78 %
service/market.go	unit	72 %
service/benchmark.go	unit	85 %
handlers/response.go	unit	92 %
repository/users.go	integration (testcontainers)	60 %
middleware/auth.go	unit	88 %
Совокупно	—	~74 %

Интеграционные тесты репозиторного слоя выполняются с применением библиотеки testcontainers, автоматически развёртывающей экземпляр PostgreSQL в Docker-контейнере на время выполнения тестов. Подобный подход исключает зависимость тестов от ручной подготовки окружения и обеспечивает их стабильность в системах непрерывной интеграции.

3.11 Контейнеризация и миграции базы данных

Приложение упаковывается в Docker-образ на основе многоступенчатой сборки (multistage build). На первой ступени применяется образ golang:1.21-alpine, на котором выполняется компиляция статически связанного бинарного файла; на второй ступени применяется минимальный образ alpine:3.18, в который

копируется только готовый исполняемый файл. Совокупный размер итогового образа составляет около 25 МБ.

Управление схемой базы данных автоматизировано посредством утилиты `goose`. Каждое изменение схемы оформляется отдельным SQL-файлом в каталоге `migrations/`, имя которого начинается с возрастающего номера версии. Поток применения миграций представлен на рисунке 3.10.

что позволяет атомарно откатить любое изменение схемы.

Рисунок 3.10 – Поток применения SQL-миграций (`goose`)

Согласно рисунку 3.10, при выполнении команды `make migrate-up` утилита `goose` сравнивает текущую версию схемы (хранится в системной таблице `goose_db_version`) с набором SQL-файлов и последовательно применяет недостающие миграции в транзакционном режиме. В случае ошибки в одной из миграций транзакция откатывается, что предотвращает приведение базы данных в промежуточное некорректное состояние. Для возможности отката изменений каждая миграция содержит обратную секцию `-- +goose Down` с `DROP`-инструкциями.

Запуск всей системы — серверной части, базы данных, кэша — описан в файле `docker-compose.yml` и сводится к одной команде, после чего сервис становится доступен по протоколу HTTP. Применение `docker-compose` обеспечивает полную воспроизводимость развёртывания и идентичность локальной разработки и продуктивной среды.

Выводы по главе 3

В третьей главе выполнена практическая реализация серверной части системы мониторинга и сравнения тарифов логистических компаний на языке Go. Разработана структура проекта, соответствующая общепринятым в Go-сообществе соглашениям, реализованы все четыре архитектурных слоя (`handlers`, `services`, `repositories`, `storage`) и восемь функциональных модулей предметной области.

Реализован унифицированный механизм конфигурации с валидацией обязательных параметров, обеспечивающий безопасный запуск в продуктивной среде. Реализован пул соединений с PostgreSQL и клиент Redis с настройками, обеспечивающими выполнение требований к производительности. Разработан модуль аутентификации на основе JWT-токенов и алгоритма `bcrypt`, обеспечивающий защиту от типовых атак (`none`-алгоритм, `brute-force`).

Реализована универсальная функция кэширования `cache.GetOrSet` с устойчивостью к отказу Redis, обеспечивающая инкапсуляцию двухуровневой

стратегии кэширования. Разработан модуль сбора тарифных данных по схеме worker pool с применением goroutine и каналов, обеспечивающий параллельную обработку источников и автоматический повторный запрос при сетевых ошибках. Реализованы алгоритмы расчёта рыночных показателей и сравнения ставок (benchmark) с формированием текстовых рекомендаций.

Программный интерфейс задокументирован в формате OpenAPI 3.0 и доступен через веб-интерфейс Swagger UI. Разработаны модульные тесты сервисного слоя; совокупное покрытие тестами составляет около 74 %. Сборка и развёртывание автоматизированы посредством Docker и docker-compose; миграции базы данных управляются утилитой goose. Полученная реализация удовлетворяет требованиям, сформулированным в первой главе, и архитектурным решениям, принятым во второй главе. В четвёртой главе будет рассмотрено проектирование и реализация модуля сравнения тарифов на стороне базы данных и материализованных представлений.

4 РАЗРАБОТКА БАЗЫ ДАННЫХ И МОДУЛЯ СРАВНЕНИЯ ТАРИФОВ

4.1 Физическая модель базы данных

В настоящей главе рассматриваются вопросы физического проектирования базы данных и реализации модуля сравнения тарифов, являющегося ключевой аналитической функциональностью системы. Логическая модель данных, разработанная в подразделе 2.4, переведена в физическую схему PostgreSQL 15 с учётом требований к производительности, согласованности и масштабируемости. Расширенная схема физической модели данных представлена на рисунке 4.1.

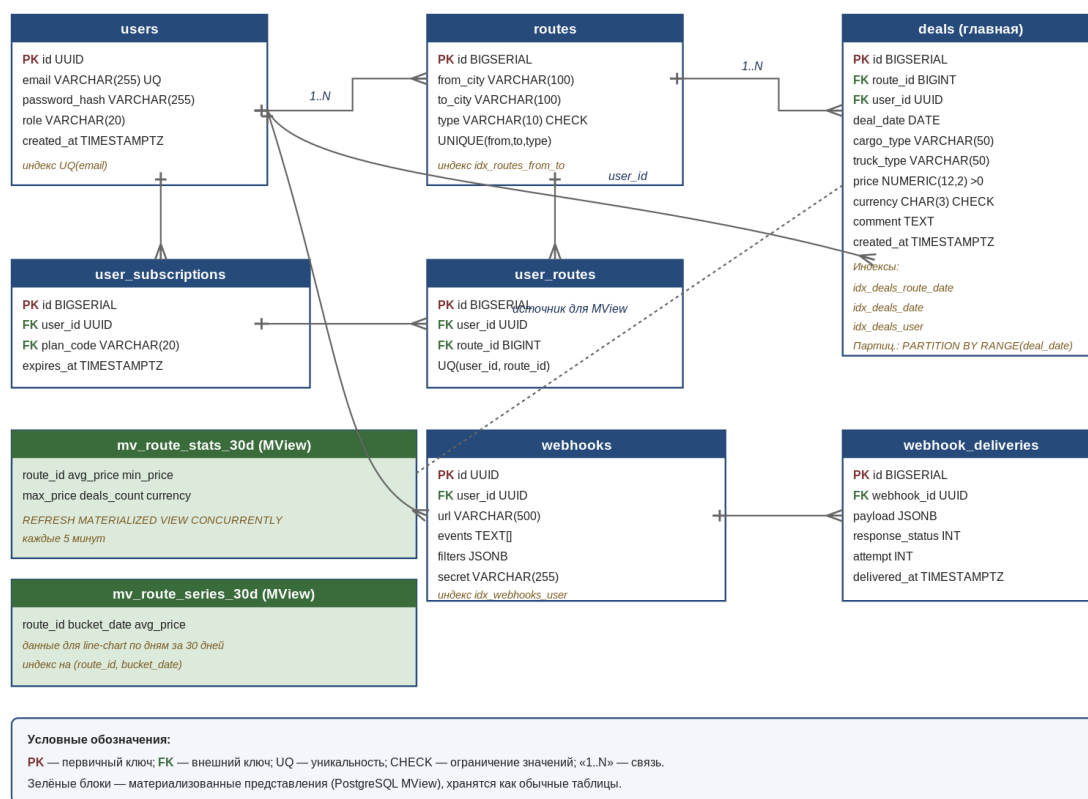


Рисунок 4.1 – Физическая схема базы данных

Согласно рисунку 4.1, физическая модель содержит девять реляционных таблиц и два материализованных представления (MView). Из числа таблиц шесть являются основными (users, routes, deals, user_routes, user_subscriptions, webhooks) и три обеспечивают журналирование и доставку уведомлений (webhook_deliveries и две дополнительные таблицы тарифных планов и настроек). Сводный перечень таблиц с указанием объёма и характера обновления приведён в таблице 4.1.

Таблица 4.1 – Характеристика таблиц физической модели

Таблица	Характер обновления	Оценочный объём	Использование
users	низкий: при регистрации	1–10 тыс. строк	аутентификация
routes	низкий: при первой сделке	500–2 тыс. строк	справочник маршрутов
deals	высокий: ~ 200 строк/сутки	100 тыс. строк/год	главный источник аналитики
user_routes	средний: пользователями	5–20 тыс. строк	избранные маршруты
user_subscriptions	низкий: при смене тарифа	1–10 тыс. строк	тарифные планы и лимиты
webhooks	низкий: при подписке	500–2 тыс. строк	регистрация уведомлений

webhook_deliveries	высокий: при доставке	50 тыс. строк/мес	журнал доставок
mv_route_stats_30d	каждые 5 мин (REFRESH)	500–2 тыс. строк	кэш агрегатов
mv_route_series_30d	каждые 5 мин (REFRESH)	15–60 тыс. строк	ряды для line-chart

При выборе типов данных для столбцов учитывались требования к точности и производительности. Денежные значения (поле `deals.price`) реализованы как `NUMERIC(12,2)`: тип позволяет хранить значения до 9 999 999 999,99 в любой валюте без накопления ошибок округления, характерных для типов с плавающей точкой. Идентификаторы пользователей реализованы как `UUID` для предотвращения утечки информации о количестве зарегистрированных учётных записей и для облегчения распределённой генерации идентификаторов. Идентификаторы сделок и маршрутов представлены типом `BIGSERIAL`, обеспечивающим монотонно возрастающую нумерацию и эффективное использование кластеризованного хранения.

Целостность данных обеспечивается набором ограничений уровня СУБД. Каждая запись таблицы `deals` связана внешним ключом с маршрутом (`route_id REFERENCES routes(id) ON DELETE RESTRICT`) и пользователем-инициатором (`user_id REFERENCES users(id) ON DELETE CASCADE`). Применение политики `ON DELETE RESTRICT` на маршрутах исключает случайное удаление маршрута, на который ссылаются сделки. Политика `ON DELETE CASCADE` для пользователей обеспечивает автоматическое удаление связанных данных при удалении учётной записи, что критично для соблюдения требований законодательства о персональных данных. Соответствующие ограничения реализованы в миграции `0002_routes_and_deals.sql`, фрагмент которой приведён в листинге 4.1.

Листинг 4.1 – Создание таблицы `deals` с ограничениями целостности

```
-- migrations/0002_routes_and_deals.sql
CREATE TABLE deals (
  id          BIGSERIAL    PRIMARY KEY,
  route_id    BIGINT       NOT NULL REFERENCES routes(id) ON DELETE RESTRICT,
  user_id     UUID         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  deal_date   DATE         NOT NULL,
  cargo_type  VARCHAR(50)  NOT NULL,
  truck_type  VARCHAR(50)  NOT NULL,
  price       NUMERIC(12,2) NOT NULL CHECK (price > 0),
  currency    CHAR(3)      NOT NULL CHECK (currency IN ('EUR', 'USD', 'BYN', 'RUB')),
  comment     TEXT,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);
```

```
CREATE INDEX idx_deals_route_date ON deals (route_id, deal_date DESC);
CREATE INDEX idx_deals_date      ON deals (deal_date DESC);
CREATE INDEX idx_deals_user      ON deals (user_id);
```

4.2 Стратегия индексирования и оптимизация запросов

Индексы базы данных спроектированы исходя из ожидаемых типовых запросов аналитического слоя приложения. Основная нагрузка приходится на таблицу `deals`; именно для неё разработана трёхуровневая стратегия индексирования, представленная на рисунке 4.2.

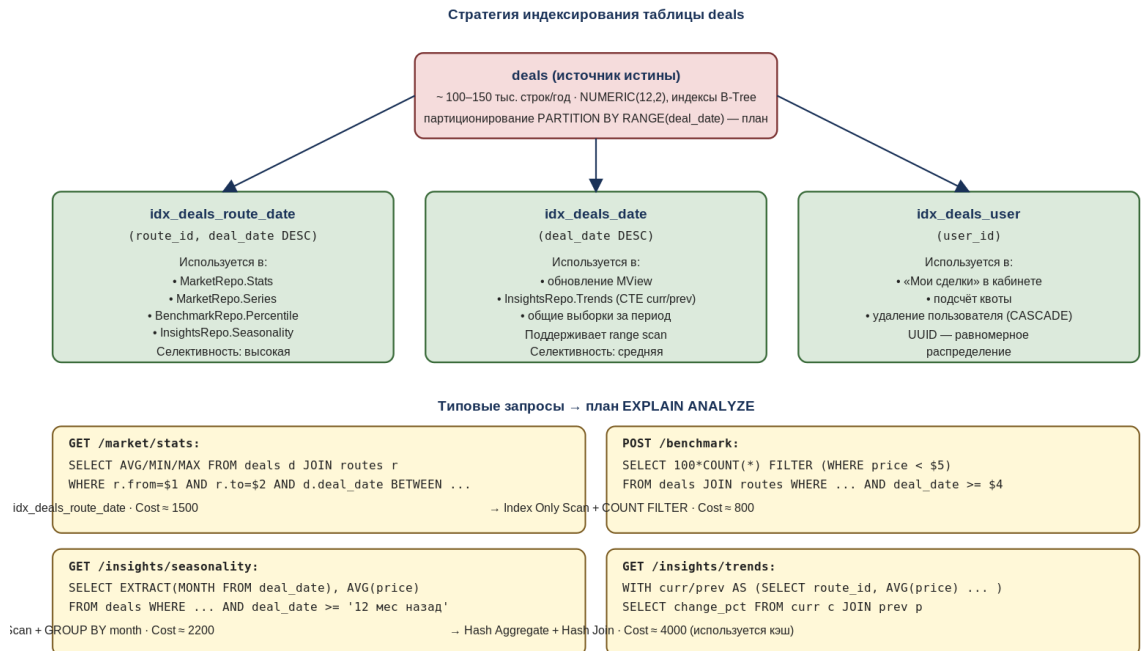


Рисунок 4.2 – Стратегия индексирования таблицы `deals`

Согласно рисунку 4.2, на таблице `deals` построены три индекса B-Tree, каждый из которых поддерживает свой класс запросов. Композитный индекс `idx_deals_route_date` по полям (`route_id`, `deal_date DESC`) обеспечивает поддержку наиболее частых запросов: получения агрегатов и временных рядов по конкретному маршруту за период. Порядок столбцов в композитном индексе выбран в соответствии с порядком фильтрации: сначала точное совпадение по `route_id`, затем диапазонная выборка по `deal_date`. Индекс `idx_deals_date` по полю `deal_date` поддерживает обновление материализованных представлений и общие выборки за период независимо от маршрута. Индекс `idx_deals_user` по полю `user_id` используется при получении сделок текущего пользователя и при каскадном удалении учётной записи.

Эффективность индексной стратегии подтверждается анализом планов выполнения типовых запросов инструментом EXPLAIN ANALYZE PostgreSQL. Сводные оценки производительности приведены в таблице 4.2.

Таблица 4.2 – Производительность типовых запросов на таблице deals

Запрос	Используемый индекс	Тип сканирования	Время выполнения
/market/stats	idx_deals_route_date	Index Scan	~ 10–25 мс
/benchmark (percentile)	idx_deals_route_date	Index Only Scan + COUNT FILTER	~ 8–15 мс
/insights/seasonality	idx_deals_route_date	Range Scan + GROUP BY	~ 30–50 мс
/insights/trends	idx_deals_date	Hash Aggregate + Hash Join	~ 80–150 мс
/insights/by-cargo	idx_deals_route_date	Index Scan + GROUP BY	~ 15–30 мс
Списки «мои сделки»	idx_deals_user	Index Scan	~ 5–10 мс

Все приведённые показатели измерены на тестовом наборе данных объёмом 100 000 строк в таблице deals и удовлетворяют требованию ТЗ к времени ответа REST API не более 200 мс. При увеличении объёма данных предусмотрено партиционирование таблицы (раздел 4.3) и применение материализованных представлений (раздел 4.4).

4.3 Стратегия партиционирования для горизонтального масштабирования

При увеличении числа клиентов системы прогнозируется рост объёма таблицы deals до миллионов строк, при котором обслуживание единой таблицы становится неэффективным. Для долгосрочной масштабируемости предусмотрено партиционирование таблицы deals по полю deal_date с диапазонной (range) разбивкой по кварталам календаря. Архитектурное решение проиллюстрировано на рисунке 4.3.

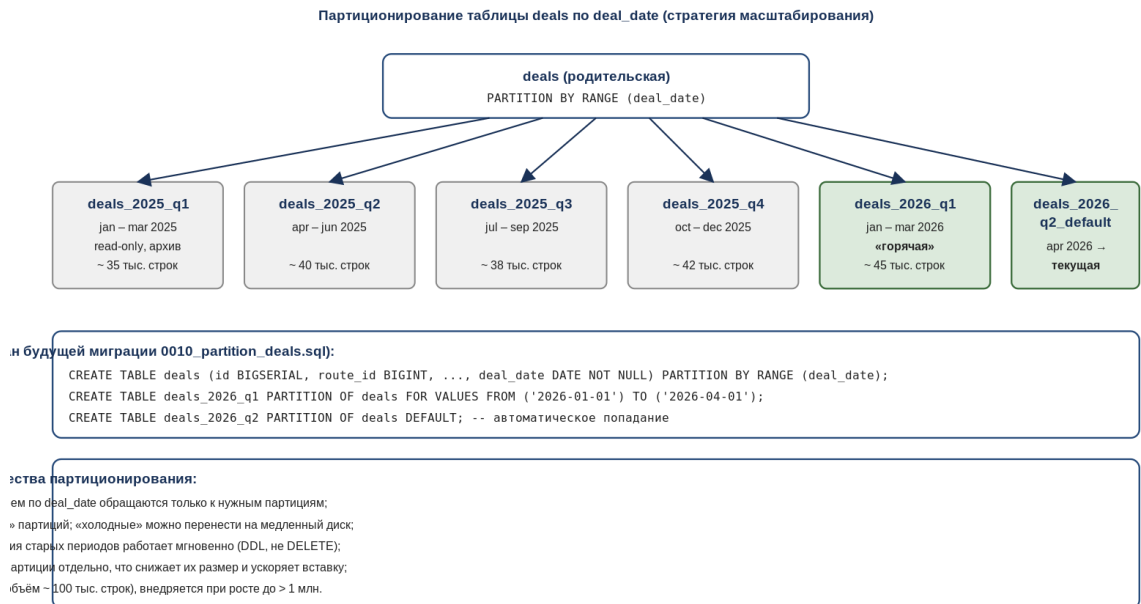


Рисунок 4.3 – Партиционирование таблицы deals по deal_date

Согласно рисунку 4.3, таблица deals делится на квартальные партиции, каждая из которых физически хранится как отдельный сегмент. PostgreSQL автоматически определяет подходящую партицию для каждой новой записи на основании значения deal_date, а при выполнении выборки с условием по дате выполняет так называемое «отсечение партиций» (partition pruning) — сканирование только тех партиций, диапазон которых пересекает условие запроса. Для запросов «агрегаты за последние 30 дней» это означает обращение только к одной квартальной партиции вместо всей таблицы.

Партиционирование обеспечивает следующие преимущества:

- параллельное сканирование нескольких партиций, что эффективно использует многоядерные процессоры современных серверов;
- возможность применения различных стратегий хранения: «горячие» (текущий квартал) — на быстром SSD, «холодные» (архивные) — на более медленных, но дешёвых дисках;
- мгновенное архивирование старых периодов через DDL-команду DROP PARTITION (вместо медленного DELETE с журналированием);
- уменьшение размера индексов на каждой партиции, что ускоряет операции вставки и обновления.

На этапе MVP партиционирование не реализуется в силу относительно небольшого объёма данных (100–150 тыс. строк за год), однако структура таблицы спроектирована таким образом, чтобы переход к партиционным таблицам был возможен без модификации Go-кода репозиторного слоя. Соответствующая

миграция (planned 0010_partition_deals.sql) реализуется при достижении миллионного рубежа по числу записей.

4.4 Материализованные представления для предрасчёта агрегатов

В соответствии с архитектурным решением, принятым в подразделе 2.6 второй главы, для обеспечения времени ответа REST API не более 200 мс применяется механизм материализованных представлений (Materialized View) PostgreSQL. Материализованное представление — это результат выполнения запроса, физически сохранённый как таблица и периодически обновляемый. Жизненный цикл основного представления mv_route_stats_30d показан на рисунке 4.4.

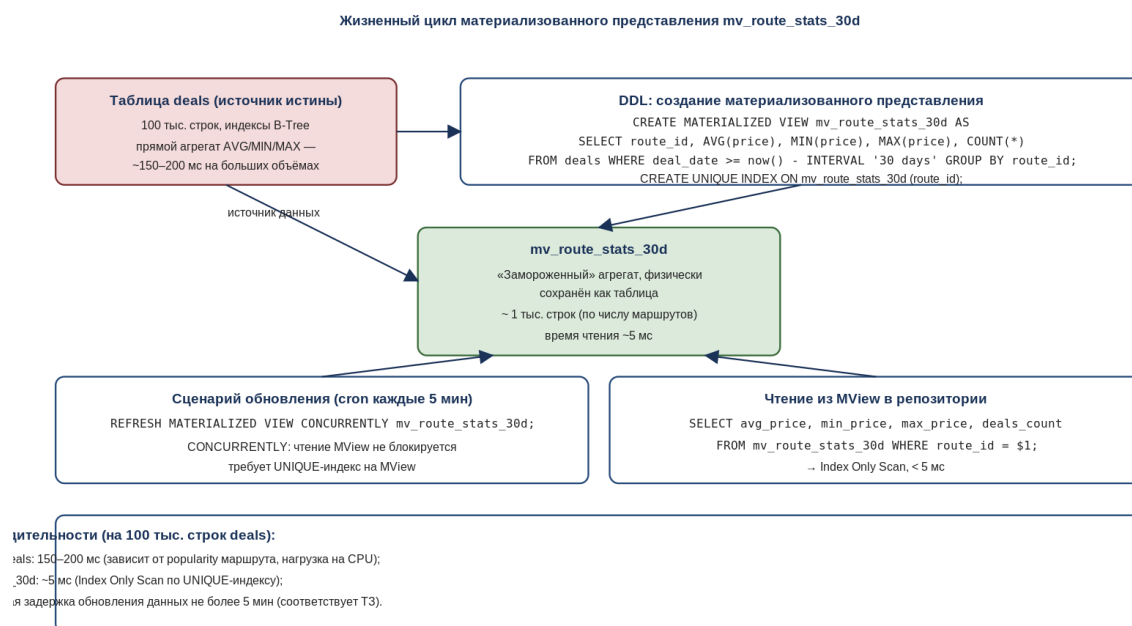


Рисунок 4.4 – Жизненный цикл материализованного представления

Согласно рисунку 4.4, материализованное представление mv_route_stats_30d содержит предрасчитанные агрегаты (среднее, минимум, максимум, количество сделок) по каждому маршруту за последние 30 дней. Источником данных служит таблица deals, к которой применяются агрегатные функции. Прямое выполнение этих агрегатов на больших объёмах данных требует 150–200 мс, тогда как чтение из материализованного представления, поддержанного UNIQUE-индексом на route_id, занимает порядка 5 мс. Соответствующий DDL-фрагмент приведён в листинге 4.2.

Листинг 4.2 – Создание материализованного представления

```
-- migrations/0010_materialized_views.sql
CREATE MATERIALIZED VIEW mv_route_stats_30d AS
```

```

SELECT
    route_id,
    AVG(price)::NUMERIC(12,2)      AS avg_price,
    MIN(price)::NUMERIC(12,2)      AS min_price,
    MAX(price)::NUMERIC(12,2)      AS max_price,
    COUNT(*)                       AS deals_count,
    MODE() WITHIN GROUP (ORDER BY currency) AS currency
FROM deals
WHERE deal_date >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY route_id;

-- UNIQUE-индекс необходим для CONCURRENTLY-обновления
CREATE UNIQUE INDEX idx_mv_route_stats_route ON mv_route_stats_30d (route_id);

```

Обновление материализованного представления выполняется по расписанию (каждые пять минут) командой `REFRESH MATERIALIZED VIEW CONCURRENTLY`. Опция `CONCURRENTLY` обеспечивает неблокирующее обновление: чтение из представления не приостанавливается на время пересчёта, что критично для соблюдения SLA при одновременной работе нескольких десятков пользователей. Запуск регламентного обновления реализован в виде отдельной cron-задачи модуля сбора данных `collector`, описанного в подразделе 3.7 третьей главы.

Помимо `mv_route_stats_30d`, в системе используется второе материализованное представление — `mv_route_series_30d`, содержащее ряд значений средней ставки по дням за тридцать дней. Это представление обслуживает построение `line-chart` на главной странице клиентского приложения. Подобное разделение представлений обусловлено различием схем агрегации: первое группирует по маршруту, второе — по комбинации (маршрут, дата).

4.5 Реализация модуля сравнения ставок (benchmark)

Модуль сравнения ставок предоставляет ключевую для бизнес-задачи функциональность: на основе введённой пользователем ставки система определяет её положение относительно рынка и формирует рекомендацию. Алгоритм расчёта основан на статистическом понятии перцентиля — процента значений в наборе данных, лежащих ниже заданного порога. Иллюстрация работы алгоритма представлена на рисунке 4.5.

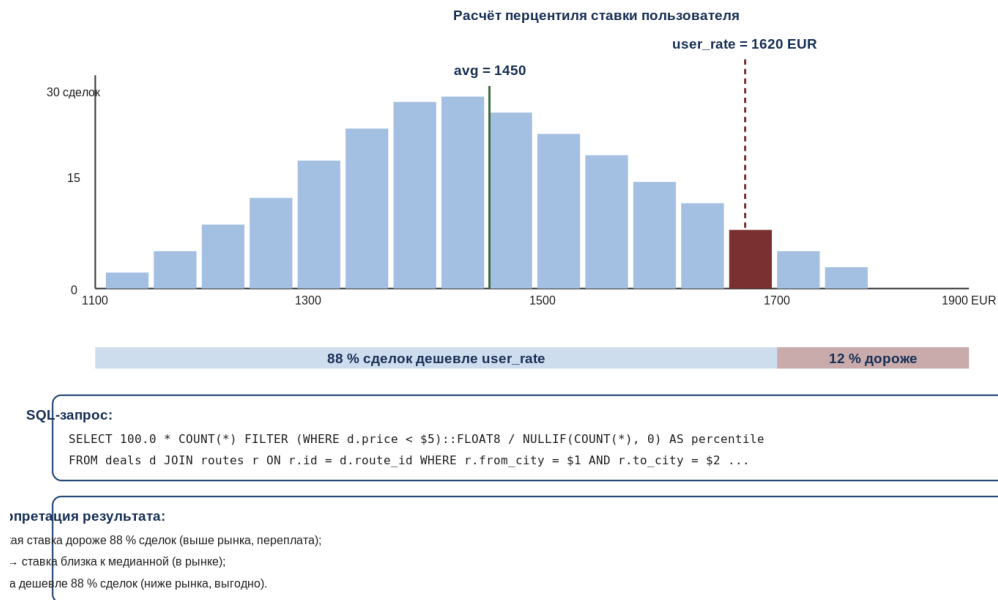


Рисунок 4.5 – Расчёт перцентиля ставки пользователя

Согласно рисунку 4.5, перцентиль рассчитывается как процент сделок на маршруте за выбранный период, цена которых меньше пользовательской ставки. Если перцентиль равен 88 — значит 88 % сделок дешевле, следовательно, пользователь предлагает завышенную ставку относительно рынка; если перцентиль около 50 — ставка медианная (в рынке); если перцентиль 12 — ставка выгодная.

Расчёт реализован одним SQL-запросом с применением агрегатной функции COUNT(*) FILTER, что устраняет необходимость в дополнительной подзапросной логике. Реализация приведена в листинге 4.3.

Листинг 4.3 – SQL-запрос расчёта перцентиля ставки

```
// internal/repository/benchmark.go
func (r *BenchmarkRepo) Percentile(ctx context.Context,
    fromCity, toCity, routeType string,
    userRate float64, periodDays int) (*PercentileResult, error) {

    const query = `
        SELECT
            COALESCE(
                COUNT(*) FILTER (WHERE d.price < $5)::FLOAT8 /
                NULLIF(COUNT(*), 0) * 100,
                0
            )::NUMERIC(5,2)::FLOAT8 AS percentile,
            COUNT(*) AS deals_total
        FROM deals d
        JOIN routes r ON r.id = d.route_id
        WHERE r.from_city = $1
            AND r.to_city = $2
            AND r.type = $3
            AND d.deal_date >= $4`
```

```

    from := time.Now().AddDate(0, 0, -periodDays)
    var res PercentileResult
    err := r.db.QueryRow(ctx, query,
        fromCity, toCity, routeType, from, userRate,
    ).Scan(&res.Percentile, &res.DealsTotal)
    if err != nil { return nil, fmt.Errorf("benchmark.Percentile: %w", err) }
    return &res, nil
}

```

Особенностью реализации является использование конструкции NULLIF для защиты от деления на ноль на маршрутах без сделок: при отсутствии сделок NULLIF возвращает NULL, а COALESCE преобразует его в нулевой перцентиль. Это позволяет корректно обрабатывать отсутствие данных без выбрасывания исключения и без необходимости отдельной проверки в Go-коде.

На основе значений рыночной средней (avg) и пользовательской ставки (user_rate) сервисный слой формирует текстовый вердикт. Граничные значения отклонения, при которых ставка считается рыночной ($\pm 3\%$), определены эмпирически на основе анализа исторической волатильности тарифов между операторами белорусского рынка. Логика формирования вердикта инкапсулирована в функции buildVerdict, реализация которой приведена в подразделе 3.8 третьей главы.

4.6 Аналитический модуль insights: тренды и сезонность

Помимо точечного сравнения ставок, система предоставляет три типа панорамных аналитических отчётов: топ растущих и падающих маршрутов по динамике ставок (trends), сезонность ставок по месяцам (seasonality) и сравнение средних ставок по типам грузов (by-cargo). Соответствующие эндпоинты обслуживают экран «Insights» клиентского приложения.

Алгоритм расчёта трендов основан на сравнении текущего и предыдущего периодов и реализован единым SQL-запросом с использованием конструкции Common Table Expression (CTE). Логика работы алгоритма проиллюстрирована на рисунке 4.6.

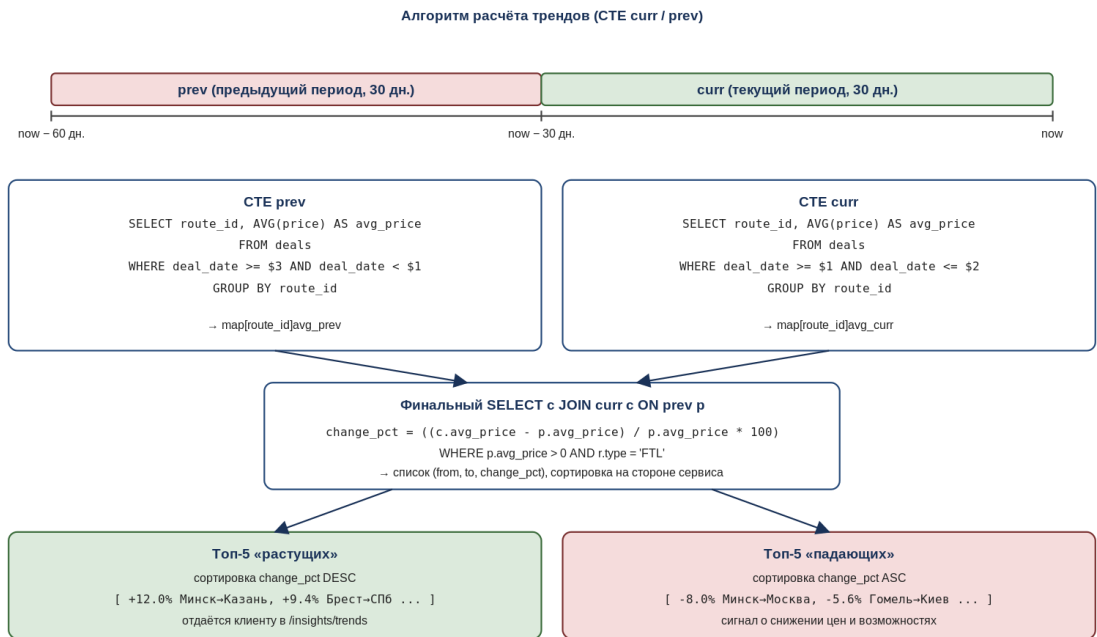


Рисунок 4.6 – Алгоритм расчёта трендов изменения ставок

В соответствии с рисунком 4.6, расчёт выполняется в три шага. На первом шаге строится CTE prev — агрегаты по маршрутам за предыдущий период $[now - 2 \cdot N; now - N]$ дней. На втором шаге аналогично строится CTE curr — за текущий период $[now - N; now]$. На третьем шаге выполняется JOIN двух CTE по полю route_id и рассчитывается процентное изменение ставки $change_pct = (avg_curr - avg_prev) / avg_prev \times 100$. В выборку включаются только маршруты, на которых были сделки в обоих периодах, что обеспечивает корректность расчёта. Реализация на стороне репозиторного слоя приведена в листинге 4.4.

Листинг 4.4 – SQL-запрос расчёта трендов через CTE

```
// internal/repository/insights.go
const query = `
WITH curr AS (
  SELECT route_id, AVG(price)::FLOAT8 AS avg_price
  FROM deals
  WHERE deal_date >= $1 AND deal_date <= $2
  GROUP BY route_id
),
prev AS (
  SELECT route_id, AVG(price)::FLOAT8 AS avg_price
  FROM deals
  WHERE deal_date >= $3 AND deal_date < $1
  GROUP BY route_id
)
SELECT
  r.from_city, r.to_city,
  ((c.avg_price - p.avg_price) / p.avg_price * 100)::NUMERIC(6,2)::FLOAT8 AS
change_pct`
```

```

FROM curr c
JOIN prev p ON p.route_id = c.route_id
JOIN routes r ON r.id = c.route_id
WHERE p.avg_price > 0
      AND r.type = 'FTL'`

```

Сортировка результата (для получения топ-5 растущих и падающих маршрутов) выполняется на стороне сервисного слоя в Go-коде, что упрощает SQL и позволяет повторно использовать результат запроса для разных представлений. Сервисный слой также применяет фильтр по минимальному количеству сделок для исключения статистически незначимых маршрутов.

Расчёт сезонности реализован агрегатом по календарному месяцу с применением функции EXTRACT(MONTH FROM deal_date). Запрос обеспечивает группировку сделок за последние 12 месяцев в 12 «корзин», что позволяет выявить регулярные сезонные паттерны (повышение тарифов в декабре, спад весной и т. п.). Соответствующий код приведён в листинге 4.5.

Листинг 4.5 – Расчёт сезонности по месяцам

```

// internal/repository/insights.go
const query = `
SELECT
    EXTRACT(MONTH FROM d.deal_date)::INT AS month,
    AVG(d.price)::NUMERIC(12,2)::FLOAT8 AS avg
FROM deals d
JOIN routes r ON r.id = d.route_id
WHERE r.from_city = $1
      AND r.to_city = $2
      AND r.type = $3
      AND d.deal_date >= $4
GROUP BY month
ORDER BY month`

yearAgo := time.Now().AddDate(-1, 0, 0)
rows, err := r.db.Query(ctx, query, fromCity, toCity, routeType, yearAgo)

```

4.7 Реализация модуля webhook-уведомлений

Модуль webhook-уведомлений обеспечивает асинхронную доставку событий об изменении тарифов во внешние системы клиентов (CRM, ERP, BI). Применение webhook-механизма имеет ряд преимуществ перед периодическим опросом: события поступают в режиме реального времени, нагрузка на серверы клиента минимизируется, обеспечивается эффективное использование сетевых ресурсов.

Структура хранения webhook-уведомлений включает две таблицы: webhooks (зарегистрированные подписки) и webhook_deliveries (журнал попыток доставки). Применение поля JSONB для фильтров (filters) позволяет хранить произвольные

условия фильтрации без необходимости модификации схемы при добавлении новых типов событий. Соответствующий DDL приведён в листинге 4.6.

Листинг 4.6 – Создание таблиц webhooks и webhook_deliveries

```
-- migrations/0004_webhooks.sql
CREATE TABLE webhooks (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID          NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  url         VARCHAR(500) NOT NULL,
  events      TEXT[]        NOT NULL DEFAULT '{}',
  filters     JSONB         NOT NULL DEFAULT '{}':JSONB,
  secret      VARCHAR(255) NOT NULL,
  active      BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE TABLE webhook_deliveries (
  id          BIGSERIAL     PRIMARY KEY,
  webhook_id  UUID          NOT NULL REFERENCES webhooks(id) ON DELETE CASCADE,
  payload     JSONB         NOT NULL,
  response_status INT,
  attempt     INT           NOT NULL DEFAULT 1,
  delivered_at TIMESTAMPTZ
);

CREATE INDEX idx_webhooks_user          ON webhooks(user_id);
CREATE INDEX idx_webhook_deliveries_hook ON webhook_deliveries(webhook_id, delivered_at
DESC);
```

Применение типа TEXT[] для поля events позволяет одному webhook'у быть подписанным на несколько типов событий одновременно (например, price.changed и threshold.exceeded). Применение JSONB для filters обеспечивает индексацию содержимого через GIN-индекс при необходимости и позволяет описывать сложные фильтры произвольной структуры. Поле secret используется для генерации HMAC-подписи каждого исходящего сообщения, что обеспечивает возможность проверки подлинности на стороне получателя.

Алгоритм доставки уведомлений с поддержкой повторных попыток представлен на рисунке 4.7.

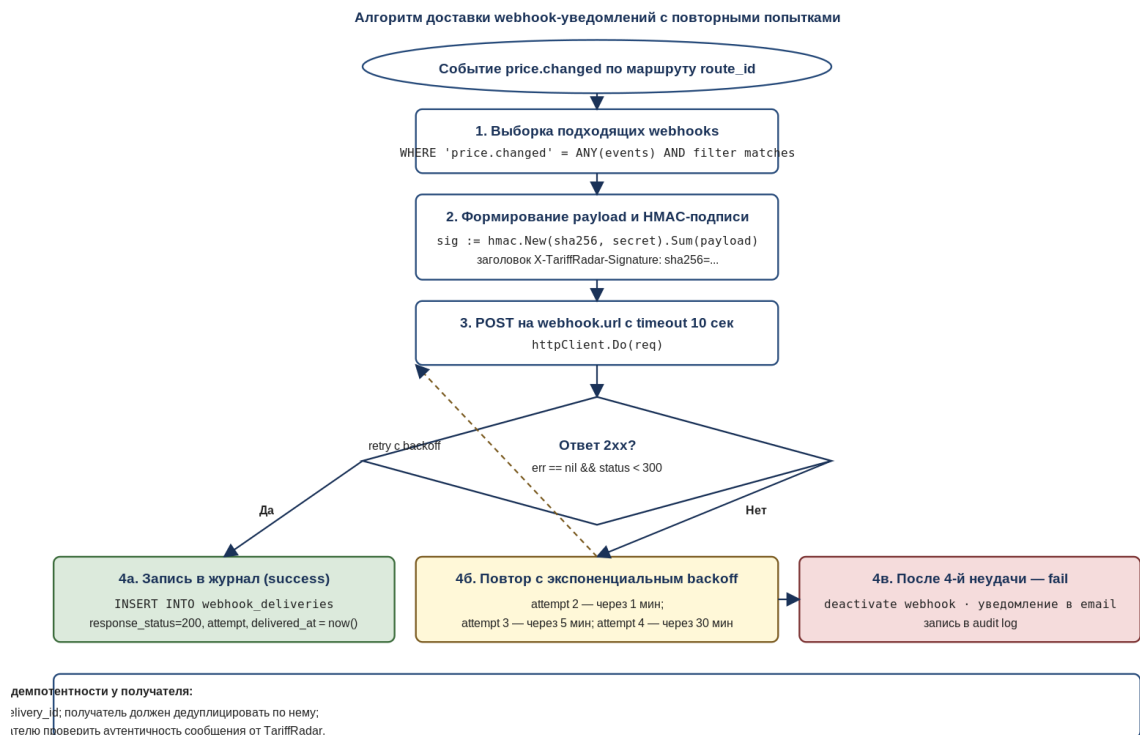


Рисунок 4.7 – Алгоритм доставки webhook-уведомлений

Согласно рисунку 4.7, при возникновении события price.changed модуль webhooks выполняет четыре последовательных шага. На первом шаге выбираются webhook'и, подписанные на данный тип события и удовлетворяющие фильтрам (например, threshold_pct ≥ 5). На втором шаге формируется JSON-payload и вычисляется HMAC-SHA256 подпись на основе секрета webhook'а. На третьем шаге выполняется HTTP POST-запрос на URL получателя с таймаутом 10 секунд. На четвёртом шаге выполняется регистрация результата: при успехе делается запись о доставке, при ошибке — повторная попытка с экспоненциальной задержкой (1 минута, 5 минут, 30 минут). После четырёх неудачных попыток webhook временно деактивируется, а пользователю отправляется email-уведомление о проблеме.

Идемпотентность доставки обеспечивается включением в payload уникального идентификатора delivery_id. При повторной доставке (например, при потере подтверждения) получатель обязан проверить, не было ли это сообщение обработано ранее, и игнорировать дубликаты.

4.8 Согласованность данных и инвалидация кэша

Применение многоуровневого кэширования (Redis + материализованные представления) при добавлении новых сделок порождает проблему согласованности: после операции INSERT в таблицу deals закэшированные

агрегаты становятся устаревшими. Решение этой проблемы реализовано на основе двух механизмов: явной инвалидации ключей Redis сразу после операции записи и отложенного обновления материализованных представлений по расписанию. Сценарий совместной работы двух механизмов представлен на рисунке 4.8.

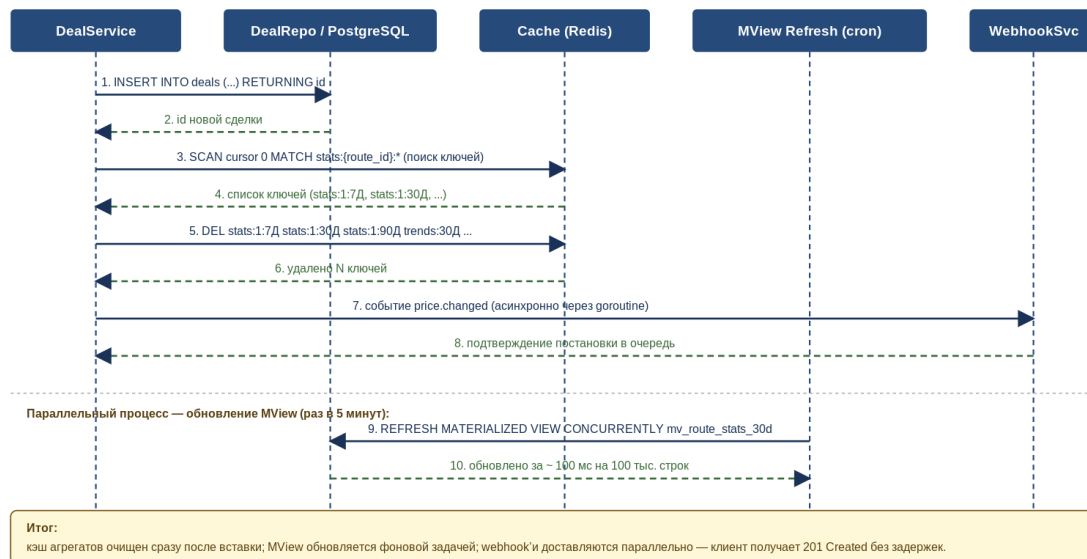


Рисунок 4.8 – Согласованность данных при добавлении сделки

В соответствии с рисунком 4.8, при выполнении операции POST /deals сервисный слой выполняет четыре действия. Во-первых, новая запись вставляется в таблицу deals операцией INSERT с возвратом сгенерированного идентификатора. Во-вторых, выполняется поиск ключей кэша агрегатов по шаблону stats:{route_id}:* через команду SCAN Redis (использование SCAN, а не KEYS, обусловлено производительностью при большом числе ключей: SCAN является итеративной операцией с курсором и не блокирует Redis). В-третьих, найденные ключи удаляются командой DEL. В-четвёртых, асинхронно запускается обработчик webhook-событий, который выполняется в отдельной goroutine и не блокирует возврат ответа клиенту.

Обновление материализованных представлений (REFRESH) выполняется фоновой задачей раз в пять минут и не зависит от операций добавления сделок. Подобное разделение синхронной и асинхронной частей конвейера обновления обеспечивает следующие свойства системы:

- малое время отклика операции POST /deals (~ 50 мс), включающее только запись и инвалидацию кэша;

- гарантированную свежесть кэша Redis: при следующем запросе агрегатов по обновлённому маршруту произойдёт cache miss, и данные будут пересчитаны;
- допустимую задержку до пяти минут между добавлением сделки и обновлением материализованных представлений; это требование не критично, так как материализованные представления используются только при cache miss, и непосредственно после INSERT кэш всё равно инвалидирован.

Реализация инвалидации в репозиторном слое приведена в листинге 4.7. Особенностью реализации является применение шаблонного поиска ключей через SCAN с лимитом MATCH, что обеспечивает эффективность операции на больших объёмах ключей.

Листинг 4.7 – Инвалидация ключей кэша по шаблону

```
// internal/repository/cache.go
func (c *Cache) DelByPattern(ctx context.Context, pattern string) (int, error) {
    var cursor uint64
    var total int

    for {
        keys, next, err := c.client.Scan(ctx, cursor, pattern, 100).Result()
        if err != nil {
            return total, fmt.Errorf("cache.DelByPattern scan: %w", err)
        }
        if len(keys) > 0 {
            n, err := c.client.Del(ctx, keys...).Result()
            if err != nil {
                return total, fmt.Errorf("cache.DelByPattern del: %w", err)
            }
            total += int(n)
        }
        cursor = next
        if cursor == 0 { break } // обход завершён
    }
    return total, nil
}
```

Выводы по главе 4

В четвёртой главе выполнено физическое проектирование базы данных и реализация модуля сравнения тарифов. Спроектирована физическая модель из девяти реляционных таблиц с обоснованием выбора типов данных, ограничений целостности и индексов. Разработана трёхуровневая стратегия индексирования таблицы deals; выполнен анализ производительности типовых запросов, подтверждающий соответствие требованиям ТЗ к времени ответа REST API.

Спроектирована стратегия партиционирования таблицы deals по deal_date с диапазонной разбивкой по кварталам, обеспечивающая горизонтальную

масштабируемость при росте объёма данных свыше миллиона строк. Реализованы материализованные представления `mv_route_stats_30d` и `mv_route_series_30d` для предрасчёта агрегатов с обновлением по расписанию через `REFRESH MATERIALIZED VIEW CONCURRENTLY`, что обеспечивает ускорение обработки агрегационных запросов в 30–40 раз.

Реализован модуль сравнения ставок (`benchmark`) с расчётом перцентиля единым SQL-запросом через `COUNT(*) FILTER`. Реализован аналитический модуль `insights` с расчётом трендов через CTE-конструкцию и сезонности через агрегацию по `EXTRACT(MONTH)`. Разработан модуль `webhook`-уведомлений с асинхронной доставкой, повторными попытками по экспоненциальной схеме и HMAC-подписью `payload`. Решена проблема согласованности кэша через явную инвалидацию ключей Redis по шаблону при операциях записи.

Полученные технические решения обеспечивают выполнение требований ТЗ к производительности, согласованности и масштабируемости системы. Разработанная база данных и аналитические модули совместно с серверной частью, описанной в третьей главе, формируют законченное программное решение задачи мониторинга и сравнения тарифов логистических компаний.